

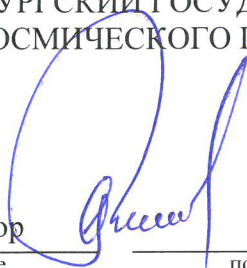
МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

ДОПУСТИТЬ К ЗАЩИТЕ

Заведующий кафедрой № 43

д-р техн. наук, профессор

должность, уч. степень, звание



06.06.2025

подпись, дата

М.Ю. Охтилев

инициалы, фамилия

БАКАЛАВРСКАЯ РАБОТА

на тему Разработка системы корпоративного телевидения для организаций

выполнена

Кузьминым Иваном Александровичем

фамилия, имя, отчество студента в творительном падеже

по направлению подготовки

09.03.04

код

Программная инженерия

наименование направления

направленности

наименование направления

02

код

Проектирование программных систем

наименование направленности

наименование направленности

Студент группы №

Z0431



06.06.2025

подпись, дата

И.А. Кузьмин

инициалы, фамилия

Руководитель

доцент, канд. техн. наук,

должность, уч. степень, звание



06.06.2025

подпись, дата

А.В. Фомин

инициалы, фамилия

Санкт-Петербург 2025

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

УТВЕРЖДАЮ

Заведующий кафедрой №43

д-р техн. наук, профессор

должность, уч. степень, звание

подпись, дата

М.Ю. Охтилев

инициалы, фамилия

ЗАДАНИЕ НА ВЫПОЛНЕНИЕ БАКАЛАВРСКОЙ РАБОТЫ

студенту группы

Z0431

номер

Кузьмину Ивану Александровичу

фамилия, имя, отчество

на тему

Разработка системы корпоративного телевидения для организаций

утвержденную приказом ГУАП от

28.03.2025

№ 11-390/25

Цель работы:

Разработка клиент-серверного Windows-приложения для централизованного управления и трансляции корпоративного контента на экранах с возможностью гибкой настройки расписания.

Задачи, подлежащие решению:

Разработать клиент-серверное приложение, реализовать механизм безопасной аутентификации и авторизации пользователей, создать модуль управления контентом, реализовать плеер для воспроизведения контента.

Содержание работы (основные разделы):

Анализ предметной области, разработка требований к программе, разработка программы, тестирование.

Срок сдачи работы « 06 »

июня

2025

Руководитель

доцент, к.т.н.

должность, уч. степень, звание

подпись, дата

А.В. Фомин

инициалы, фамилия

Задание принял(а) к исполнению

студент группы №

Z0431

И.А. Кузьмин

РЕФЕРАТ

Выпускная квалификационная работа, 114 страниц, 30 рисунков, 3 таблицы, 15 источников, 3 приложения.

Ключевые слова: централизованное управление контентом, трансляция видео, настройка расписания, корпоративная культура

Актуальность: Актуальность данной темы обусловлена неэффективностью стандартных корпоративных коммуникаций, технологическим устареванием существующих платформ, а также необходимостью цифровой трансформации бизнес-процессов и поддержки удаленных форматов работы.

Цель работы: разработка современной платформы для корпоративного телевидения, позволяющей эффективно управлять внутренними коммуникациями компании, повысить вовлеченность и оптимизировать процессы передачи корпоративной информации новым сотрудникам.

В процессе использовались программные средства: IDE PyCharm, QtCreator, Django, Celery, Qt Framework, FFmpeg.

Примененные языки программирования: C++, Python.

Полученные результаты: разработана система корпоративного телевидения с централизованным управлением контентом, плейлистами, расписаниями и отделами.

Область применения: внутренние коммуникации средних и крупных компаний, особенно с распределенной структурой (филиалы), наличием удаленных сотрудников, где важна скорость и эффективность донесения информации.

ABSTRACT

Final qualifying work, 114 pages, 30 figures, 3 tables, 15 sources, 3 appendices.

Keywords: centralized content management, video broadcast, schedule setting, corporate culture

Relevance: the relevance of this topic is due to the inefficiency of standard corporate communications, the technological obsolescence of existing platforms, as well as the need for digital transformation of business processes and support for remote work formats.

Purpose of the work: development of a modern platform for corporate television, which allows you to effectively manage internal communications of the company, increase involvement and optimize the processes of transmitting corporate information to new employees.

Software tools used in the process: IDE Pycharm, Qtcreator, Django, Celery, Qt Framework, FFmpeg.

Programming languages used: C ++, Python.

Results obtained: a corporate television system with a centralized content of content, playlists, schedules and departments has been developed.

Application area: internal communications of medium and large companies, especially with a distributed structure (branches) and the presence of remote employees, where speed and efficiency of information delivery are important.

СОДЕРЖАНИЕ

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	4
ВВЕДЕНИЕ	5
1 Анализ предметной области	7
1.1 Актуальность проблемы	7
1.1.1 Сравнение существующих аналогов	9
1.2 Цели и задачи разработки программы	11
2 Разработка и анализ требований	13
2.1 Функциональные требования	13
2.2 Нефункциональные требования	14
2.3 Выбор средств реализации.....	15
3 Разработка системы.....	20
3.1 Проектирование архитектуры системы	20
3.2 Описание структур и базы данных	23
3.3 Описание сервисов и окружения приложения.....	29
4 Тестирование	32
4.1 Тестирование авторизации пользователя в системе.....	32
4.2 Тестирование взаимодействия с плейлистами, назначение плейлистов на отделы и добавление расписания трансляции	34
4.3 Тестирование API.....	42
4.3 Тестирование клиентского приложения «Заставка»	45
ЗАКЛЮЧЕНИЕ	49
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	50
Приложение А	52
Приложение Б.....	53
Приложение В	95

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

API – Application Programming Interface (интерфейс программирования приложений)

REST - архитектурный стиль для создания распределенных систем

БД – база данных

HTTP – HyperText Transfer Protocol (протокол для получения ресурсов)

JSON (от англ. JavaScript object notation) – нотация объектов JavaScript

IDE (от англ. integrated development environment) – Интегрированная среда разработки

CRUD - аббревиатура, обозначающая четыре основных операции управления данными: create, read, update, delete/destroy

YAML - язык для сериализации данных

ВВЕДЕНИЕ

Цифровая трансформация бизнес-процессов выводит эффективность внутренних коммуникаций на передний план при управлении компанией. Перед организациями сегодня стоят вызовы: необходимость мгновенно доводить сведения до коллектива, усиливать включенность персонала в рабочие процессы и формировать сплоченную корпоративную культуру.

Классические инструменты – будь то электронная почта, внутренние порталы или планерки проигрывают в гибкости и интерактивности. Это создает пробуксовку в передаче знаний между отделами и влияет на скорость усвоения информации сотрудниками.

Корпоративное ТВ один из элементов системы внутренних коммуникаций, работающих на задачу вовлеченности и создания мотивирующей атмосферы. Этот подход решает три задачи сразу: налаживает диалог между отделами, сокращает сроки адаптации новых сотрудников и формирует общее информационное пространство. А использование видео обеспечивает ясность и запоминаемость материалов — главный козырь в эпоху удаленки и гибридного графика.

Целью данной выпускной квалификационной работы является разработка системы корпоративного телевидения, которая позволит компаниям эффективно управлять внутренними коммуникациями, повысить вовлеченность сотрудников и оптимизировать процессы передачи информации. В рамках проекта планируется реализовать функционал для трансляции видеоматериалов и персонализации контента в зависимости от роли сотрудника.

Практическая значимость проекта заключается в том, что внедрение такой системы позволит компаниям:

1. Операционные преимущества:

- сократить время доведения информации,
- повысить вовлеченность сотрудников,
- уменьшить нагрузку на стандартные каналы коммуникаций.

2. Экономические выгоды:

- снизить затраты на внутренние коммуникации,
- сократить расходы на обучение персонала.

3. Стратегические преимущества:

- укрепить корпоративную культуру,
- повысить прозрачность бизнес-процессов,
- создать единое цифровое пространство для обмена знаниями.

Таким образом, разрабатываемая система корпоративного телевидения представляет собой не просто технологическое решение, а стратегический инструмент трансформации внутренних коммуникаций компании. Она позволяет перейти от фрагментированных, неэффективных каналов передачи информации к целостной, интерактивной медиа-среде, которая способствует повышению вовлеченности сотрудников, укреплению корпоративной культуры и, в конечном итоге, росту производительности и эффективности бизнеса в условиях цифровой экономики.

1 Анализ предметной области

1.1 Актуальность проблемы

Корпоративное телевидение - это современная цифровая платформа для организации внутренних коммуникаций предприятия, представляющая собой интегрированную систему управления и распространения мультимедийного контента среди сотрудников и включающее в себя:

- централизованное управление видеоконтентом,
- планирование трансляций и вещания,
- интерактивные элементы взаимодействия.

Актуальность внедрения таких систем обусловлена следующими факторами:

1. Кризис стандартных коммуникаций:

- только 23% сотрудников полностью читают корпоративные рассылки [2],
- информация на собраниях усваивается лишь на 20-30% [3],
- вовлеченность персонала в корпоративные процессы не превышает 40% [4].

2. Технологические вызовы:

- устаревание существующих коммуникационных платформ,
- необходимость интеграции с современными бизнес-процессами,
- требования цифровой трансформации.

3. Организационные последствия:

- потери производительности из-за неэффективных коммуникаций достигают 15% [5],

- затраты на адаптацию нового сотрудника увеличились на 40% за последние 3 года [6],

- текучесть кадров в компаниях со слабыми коммуникациями на 25% выше среднего [6][4].

Основные проблемы существующих решений:

- сложность интеграции с российскими системами,
- отсутствие персонализации контента,
- высокая стоимость внедрения и обслуживания.

Корпоративное телевидение становится незаменимым инструментом для крупных компаний с разветвленной сетью филиалов, организаций, где часть сотрудников работает удаленно, а также для предприятий, где критически важна скорость и эффективность донесения информации до каждого члена коллектива. Оно объединяет, информирует и вовлекает, создавая единое коммуникационное пространство независимо от географической удаленности или масштабов бизнеса.

1.1.1 Сравнение существующих аналогов

На современном рынке существует множество решений для организации корпоративного телевидения, каждое из которых имеет свои особенности, преимущества и недостатки. В данном разделе будет проведен сравнительный анализ наиболее популярных платформ, которые используются для внутренних коммуникаций в компаниях.

1. «Microsoft Stream»

- функциональность - платформа позволяет загружать, хранить и транслировать видеофайлы, а также интегрируется с другими продуктами Microsoft 365;

- преимущества - удобный интерфейс, есть возможность создания плейлистов для различных отделов, наличие интеграции с Teams для проведения обучающих сессий;

- недостатки: ограниченные возможности кастомизации под специфические нужды HR-менеджеров;

2. «Vimeo Enterprise»

- функциональность: vimeo Enterprise имеет качественный видеохостинг с возможностью создания защищенных каналов для внутреннего использования;

- преимущества: высокое качество видео, есть возможность настройки контента и аналитики просмотров;

- недостатки: высокая стоимость, подписка может быть нецелесообразной для маленьких компаний;

3. «Kaltura»

- функциональность: гибко управляет видеоархивом, включая встроенные инструменты для монтажа и адаптации контента под нужды компании;

- преимущества: легко стыкуется с корпоративными HR-системами, поддерживает почти любые видеоформаты – идеально для сложных IT-экосистем;

– недостатки: первоначальная настройка часто требует помощи технических специалистов;

4. «Смотрим»

– функциональность: позволяет развернуть корпоративный видеоцентр "под ключ" – от трансляций до администрирования контента;

– преимущества: интуитивно понятный интерфейс, выгодные бизнес-тарифы, гибкость под индивидуальные задачи;

– недостатки: функционал скромнее, чем у западных платформ;

5. «Spinetix»

– функциональность: spinetix является решением для цифровых вывесок, трансляции видео на экраны в офисах и включает управление контентом через облачные технологии;

– преимущества: наличие кастомизации контента, возможность интеграции с различными источниками данных и простота использования интерфейса;

– недостатки: необходимо обучение сотрудников для эффективного использования системы;

Проведем оценку каждого приложения по выбранным критериям. Результаты проведенной оценки представлены в таблице 1.

Таблица 1 – Вывод по анализу существующих решений

Приложение	Microsoft Stream	Vimeo Enterprise	Kaltura	Смотрим (Smotrim)	Spinetix
Простота в использовании	Удобный интерфейс, легко осваивается	Средняя	Средняя	Легкий в использовании	Средняя
Аналитика	Да, базовая	Да, расширенная	Да, расширенная	Да, минимальная	Да, базовая
Русскоязычный интерфейс	Да	Да	Да	Да	Да

Продолжение таблицы 1

Интеграция с другими системами	Хорошая	Нет	Отличная	Ограниченная	Хорошая
Возможности кастомизации	Нет	Высокие	Высокие	Высокие	Высокие
Поддержка интерактивного контента	Да	Да	Да	Нет	Да

Был проведен сравнительный анализ платформ для корпоративного телевидения (Microsoft Stream, Vimeo Enterprise, Kaltura, Смотрим, Spinetix), который выявил существенные различия в их функциональных возможностях, их использовании, поддержке интерактивности и гибкости кастомизации. Каждое решение обладает уникальным набором преимуществ и ограничений, что делает выбор оптимальной платформы зависимым от конкретных задач компании, ее масштаба, ИТ-инфраструктуры, бюджета и требований HR-отдела к инструментам внутренних коммуникаций.

1.2 Цели и задачи разработки программы

Разработать современную платформу корпоративного ТВ, которая улучшит внутреннюю коммуникацию в компании и оптимизирует управление видеоконтентом. Приложение должно обеспечить сотрудникам компаний удобный доступ к актуальной информации и новостям, а также будет способствовать формированию единой корпоративной культуры.

Задача работы заключается в создании системы, которая будет включать в себя следующие функции:

- система должна обеспечивать функции загрузки, хранения и удаления видеоматериалов. Это позволит администраторам поддерживать актуальность видеоконтента и его обновление для целевой аудитории сотрудников. Сотрудники смогут получать актуальную информацию с запланированным разрывом времени.

- необходимо реализовать единую систему авторизации, чтобы пользователи могли входить в личный кабинет с разных устройств.

- менеджеры и администраторы системы должны иметь возможность настраивать ротацию контента по дням недели и времени суток. Например, обучающие видео могут быть запланированы на утренние часы, а новости компании — на обеденное время. Это обеспечит более эффективное информирование сотрудников отделов.

- интерфейс для просмотра контента. Панель управления должна обеспечить пользователям удобный интерфейс для просмотра загружаемого контента, новостей компании и других материалов, созданных администрацией или HR-менеджерами.

- представление информации на экране, а также частота демонстрации информации должны быть настроены с учетом распорядка дня сотрудников, специфики трудового процесса и локации экранов.

С помощью данных функций в системе корпоративного телевидения можно создать эффективный инструмент для внутренней коммуникации с сотрудниками, повысить уровень вовлеченности и обеспечить доступ к актуальной информации в любое время.

2 Разработка и анализ требований

В процессе создания системы корпоративного ТВ первый критически важный шаг — четко сформулировать требования к системе. Разработчики разделяют их на два типа:

- Что система должна делать (функциональные требования)
- Как она должна это делать (нефункциональные требования)

2.1 Функциональные требования

Функциональные требования представляют собой характеристики и возможности, которые система должна обеспечить для удовлетворения потребностей пользователей и достижения бизнес-целей. Эти требования описывают, как система должна функционировать в различных сценариях использования, а также определяют действия, которые должны быть выполнены разработчиками для реализации этих функций. К основным функциональным требованиям разрабатываемой системы относятся:

- управление пользователями - важный аспект системы, так как он обеспечивает безопасность и определяет порядок взаимодействия с контентом и плейлистами. Ключевые функции управления пользователями включают:

- а) возможность добавлять новых пользователей в систему с использованием уникального логина и пароля используя административную часть системы. Процесс должен быть простым и интуитивно понятным, чтобы минимизировать барьеры для администраторов;

- б) настройка профиля (смена пароля, добавление информации о должности и т.д.);

- управление контентом - основной компонент системы, он определяет, как видеоматериалы будут загружаться и транслироваться на устройствах. Ключевые функции управления контентом включают:

- а) загрузка новых и удаление существующих видеоматериалов из системы;

- б) настройка механизма ротации контента с учетом временных параметров (день недели, время суток);

в) назначение правил трансляции контента по отделам;

– просмотр контента в панели управления – важная часть системы, которая обеспечивает удобство ее использования. Ключевые функции просмотра контента включают:

- а) воспроизведение видеоматериалов, различных форматов и кодеков;
- б) поиск и фильтрация контента по названию, дате загрузки;

– режим экранной заставки является дополнительной возможностью системы, которая позволяет эффективно использовать экраны во время простоя или неактивности пользователей. Ключевые функции заставки включают:

- а) автоматическая активация и воспроизведение назначенного видеоконтента на экранах при переходе системы в режим заставки;

2.2 Нефункциональные требования

Нефункциональные требования к системе корпоративного телевидения определяют качество, производительность и другие характеристики системы, которые не связаны напрямую с её функциональностью. Соблюдение этих требований является критически важным для обеспечения стабильности, безопасности и соответствия ожиданиям пользователей.

Основные нефункциональные требования к системе корпоративного телевидения:

- эргономичность интерфейса:
 - а) пользовательский интерфейс системы должен обладать интуитивной навигацией и структурой, обеспечивающей эффективный доступ к контенту и управление настройками без необходимости дополнительного обучения;
- масштабируемость:
 - а) архитектура системы обязана обеспечивать линейное увеличение производительности при росте нагрузки (количества пользователей, объема контента) без критической деградации времени отклика;
- производительность:

а) время загрузки видеоконтента для воспроизведения не должно превышать 5 секунд при стандартном корпоративном канале связи (100 Мбит/с), а время отклика на действия пользователя (например, переключение между видео) — 1 секунду;

– адаптивность:

а) приложение должно корректно отображаться и функционировать на устройствах с различными размерами экранов;

– безопасность:

а) система должна обеспечивать надежные методы аутентификации пользователей и шифрования данных, включая хранение паролей в базе данных в виде хэша;

Реализация указанных нефункциональных требований обеспечит создание надежной системы, которая не только выполняет свои функции, но и делает это эффективно и безопасно. Способствуют повышению удовлетворенности пользователей от взаимодействия с системой.

2.3 Выбор средств реализации

Разработка системы корпоративного телевидения требует тщательного подхода к выбору технологий, так как решение должно одновременно обеспечивать надежную работу с видеоконтентом, масштабируемость под нужды крупных предприятий, удобный интерфейс для пользователей и эффективно реализовывать функциональные и нефункциональные требования.

2.3.1 Основой серверной части стал Django - мощный Python-фреймворк, который идеально подходит для подобных проектов. Его главное преимущество заключается в концепции "всё в комплекте" - разработчики получают из коробки все необходимое для создания полнофункционального веб-приложения [9]. Основные преимущества:

– чистая архитектура MVT (Model-View-Template), разделяет логику приложения на три основных компонента: модель (Model), представление

(View) и (Template) представляет логику представления в виде сгенерированной разметки html [9]. Это разделение способствует лучшей организации кода, упрощает его поддержку и тестирование,

- встроенная панель администратора позволяет быстро управлять данными приложения без необходимости разработки отдельного интерфейса,
- объемная документация и большое сообщество обеспечивает постоянное обновление фреймворка, а также наличие множества сторонних библиотек и расширений, которые могут быть интегрированы в проект,

Для реализации REST API был выбран Django REST Framework, который органично дополняет Django и предоставляет все необходимые инструменты для создания современного API [10].

Основные преимущества:

- поддержка всех стандартных HTTP-методов (GET, POST, PUT, DELETE), что позволяет легко реализовать CRUD-операции (создание, чтение, обновление и удаление) для ресурсов приложения [10],
- встроенная аутентификация и авторизация позволяет защитить API от несанкционированного доступа и управлять правами пользователей,
- сериализация данных - процесс преобразования сложных типов данных (например, объектов моделей) в формат JSON или XML для передачи по сети, что упрощает работу с API [10],
- пагинация результатов запросов оптимизирует работу с API,
- фильтрация запросов позволяет клиентским приложениям получать только те данные, которые им необходимы,

Для хранения и управления данными используется PostgreSQL. Выбор обусловлен требованиями надежности, отказоустойчивости и производительности при работе со сложными запросами [14].

Основные преимущества:

- репликация для обеспечения высокой доступности,
- эффективные механизмы индексации для ускорения поиска,
- параллельное выполнение запросов.

Для обеспечения отзывчивости системы и обработки ресурсоемких операций реализованы два ключевых компонента:

Redis - обеспечивает передачу сообщений между различными компонентами системы, что особенно важно для асинхронной обработки задач такие как конвертация видеоматериалов в единый формат. Также используется для кэширования результатов часто выполняемых запросов к базе данных, таких как список доступного контента. Это значительно снижает нагрузку на основную базу данных и ускоряет время отклика системы, позволяя избежать повторных вычислений и запросов к базе данных, что особенно важно при работе с большими объемами [15].

Основные функции Redis:

- управление данными сессий пользователей,
- кэширование часто запрашиваемых данных.

Celery — это асинхронный фреймворк для выполнения длительных операций, который интегрируется с базой данных Redis. Позволяет распределять ресурсоемкие операции по обработке видео и другими задачами на отдельные рабочие процессы.

Celery для распределенных задач:

- конвертация видеофайлов в требуемые форматы,
- индексация контента.

2.3.2 Для реализации клиентского приложения выбран фреймворк Qt6, обеспечивающий возможность работы с видео на разных платформах и высокую скорость обработки данных [12]. Основными причинами выбора являются:

- QML (Qt Modeling Language) представляет собой декларативный язык программирования, который позволяет разработчикам создавать интуитивно понятные и привлекательные пользовательские интерфейсы. [12],

- QtMultimedia обеспечивает воспроизведение видеоконтента, включая поддержку современных кодеков и форматов. Это позволяет приложению воспроизводить широкий спектр видеоматериалов без использования сторонних библиотек [12],
- QtNetwork реализует удобный интерфейс для выполнения сетевых запросов к API сервера. Это позволяет клиентскому приложению взаимодействовать с серверной частью системы, получать актуальные данные о контенте и его расписании[12],
- QSql это инструмент для работы с базами данных, включая SQLite. Это позволяет клиентскому приложению эффективно управлять локальными данными, такими как, пользовательские настройки и плейлисты [12],
- кроссплатформенность позволяет развернуть приложение как на Windows, так и на Linux без значительных изменений в коде,
- аппаратное ускорение повышает производительность приложения, снижая нагрузку на центральный процессор (CPU) и обеспечивает плавное воспроизведение даже при высоких разрешениях [12].

Для хранения данных в клиентском приложении используется SQLite — легковесная СУБД хорошо подходит для задач хранения информации о контенте и пользовательских настройках. Основные преимущества SQLite включают:

- кэширования контента,
- хранение пользовательских настроек,
- хранение локальных плейлистов.

Выбранный технологический стек зарекомендовал себя как оптимальное решение благодаря многолетней практике успешного применения - система демонстрирует стабильную работу при высокой нагрузке, обеспечивает необходимый уровень безопасности и предоставляет удобные инструменты для администраторов и конечных пользователей.

Сочетание надежности Django, производительности PostgreSQL и гибкости Qt6 создает прочную основу для развития системы в будущем.

2.3.3 Диаграмма компонентов системы корпоративного телевидения

Диаграмма компонентов системы корпоративного телевидения представляет собой визуальное отображение архитектуры системы, показывающее, как различные компоненты взаимодействуют друг с другом. Выделяются несколько ключевых компонентов, каждый из которых выполняет свою роль в обеспечении функциональности и производительности системы:

- взаимодействие осуществляется через REST API. Клиент отправляет HTTP-запросы на сервер для получения данных,
- Django выполняет SQL-запросы к PostgreSQL для получения или изменения информации о пользователях и контенте,
- Django использует Redis для кеширования часто запрашиваемых данных, что позволяет ускорить доступ к ним,
- Django отправляет задачи в очередь Redis для последующей обработки Celery,
- Celery может выполнять операции с базой данных по мере необходимости при выполнении фоновых задач,
- приложение сохраняет данные на локальный диск для повышения производительности.

Схема взаимодействия компонентов представлена на рисунке 1.

.

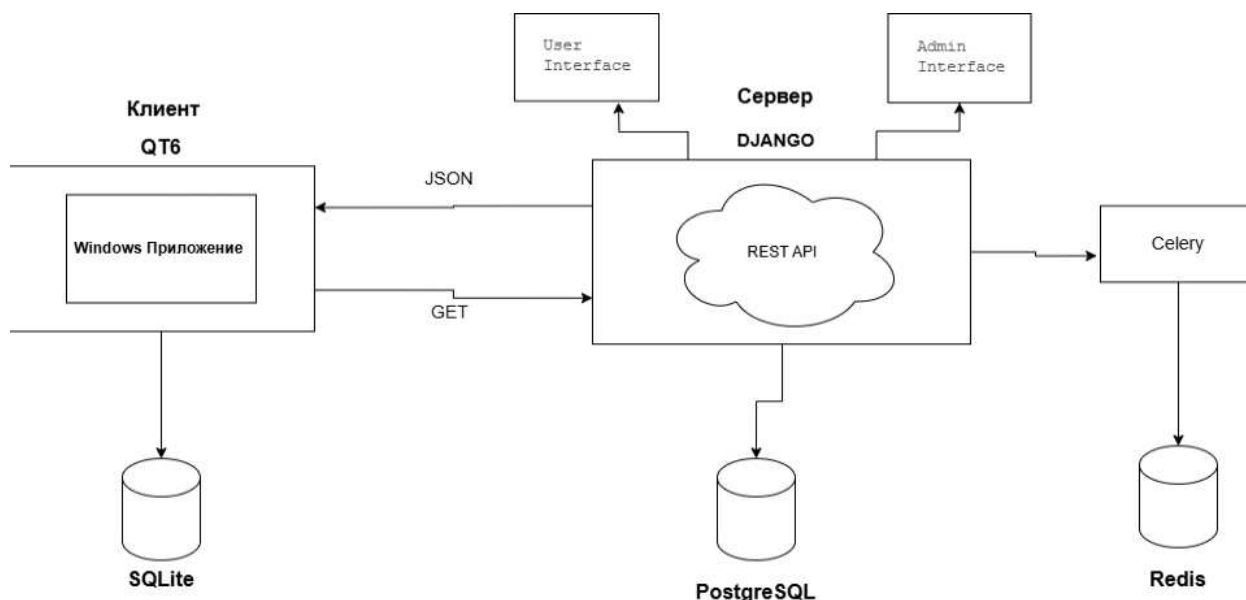


Рисунок 1. Диаграмма компонентов корпоративного телевидения

3 Разработка системы

3.1 Проектирование архитектуры системы

Разрабатываемая система корпоративного телевидения — это решение с многоуровневой структурой, которая обеспечивает чёткое распределение обязанностей между компонентами.

Клиентская часть системы разделена на два независимых модуля, взаимодействующих через IPC. Это решает проблему "зависаний" интерфейса при работе с сетью.

Фоновый сервис загрузки:

- автономно управляет всеми сетевыми операциями,
- загружает расписание трансляций и видеоконтент,
- передает расписание и список актуального контента заставке.

Приложение-заставка:

- полноэкранное воспроизведение контента,
- работа с готовым локальным контентом,
- запрос актуального контента.

Рабочий цикл приложения:

- приложение-заставка запрашивает контент через LocalSocket,
- сервис обрабатывает запрос и загружает контент с сервера,
- контент сохраняется на локальном диске,
- сервис передает путь к файлу через LocalSocket,
- приложение-заставка воспроизводит контент с локального диска.

Серверная часть реализована как единый модуль с четкой внутренней организацией. Архитектура основана на принципах MVT (Model-View-Template) - это архитектурный паттерн, который разделяет приложение на три основные компоненты [10]:

Модель (Model) - управляет данными и бизнес-логикой приложения. Модель отвечает за хранение, извлечение и обновление данных, а также за выполнение бизнес-логики.

Представление (View) - отвечает за отображение данных пользователю. Получает данные от модели и представляет их в удобном для пользователя формате, например, в виде JSON.

Шаблон (Template) - обрабатывает пользовательские запросы, взаимодействует с моделью для обновления данных и выбирает соответствующее представление для отображения. Шаблон выступает в роли посредника между моделью и представлением, передавая входные данные от пользователя модели и выбирая, какое представление показать пользователю.

Ключевые компоненты и их взаимодействие:

- модели (Models) - определяют структуру данных и бизнес-логику,
- представления (Views): обрабатывают HTTP-запросы и формируют ответы. Используют Django REST Framework (DRF) для построения API,
- URL-маршрутизатор (URLs): сопоставляет URL-адреса с представлениями,
- сериализаторы (Serializers): (DRF) преобразуют объекты моделей в JSON для ответов API. Определяют, какие поля модели включать/исключать,
- панель администратора (Django Admin): Готовая CRUD-панель для управления моделями,
- слой данных (PostgreSQL): Все структурированные данные (модели Django) хранятся в реляционных таблицах,
- слой кэширования и очередей задач (Redis): выступает в качестве кэширование HTML-фрагментов и организует очереди задач. Django помещает задачи в Redis, Celery забирает их оттуда на выполнение.

Для обмена данными между клиентом и сервером используется Representational State Transfer Application Programming Interface (REST API). REST – это архитектура программного обеспечения для взаимодействия по протоколу HTTP.

На рисунке 2 представлена диаграмма последовательности взаимодействия системы.

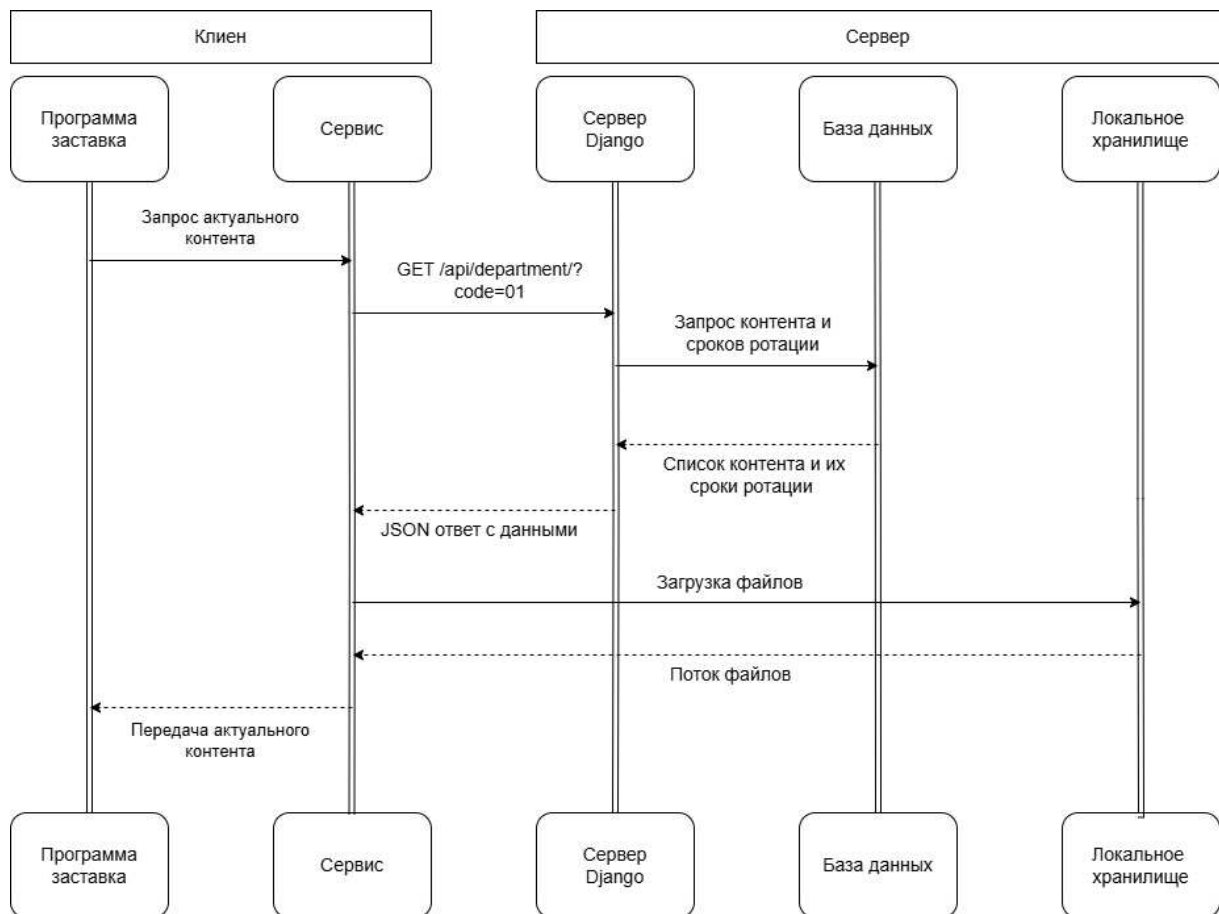


Рисунок 2 – Диаграмма последовательности взаимодействия системы

3.2 Описание структур и базы данных

Основным подходом к проектированию баз данных и информационных систем является модель "сущность-связь" (Entity-Relationship, ER). ER-диаграмма — это тип блок-схемы, который визуализирует взаимосвязи между различными сущностями в системе.

Диаграмма отношений сущностей, описывающая таблицы БД серверной части приведена на рисунке 3.

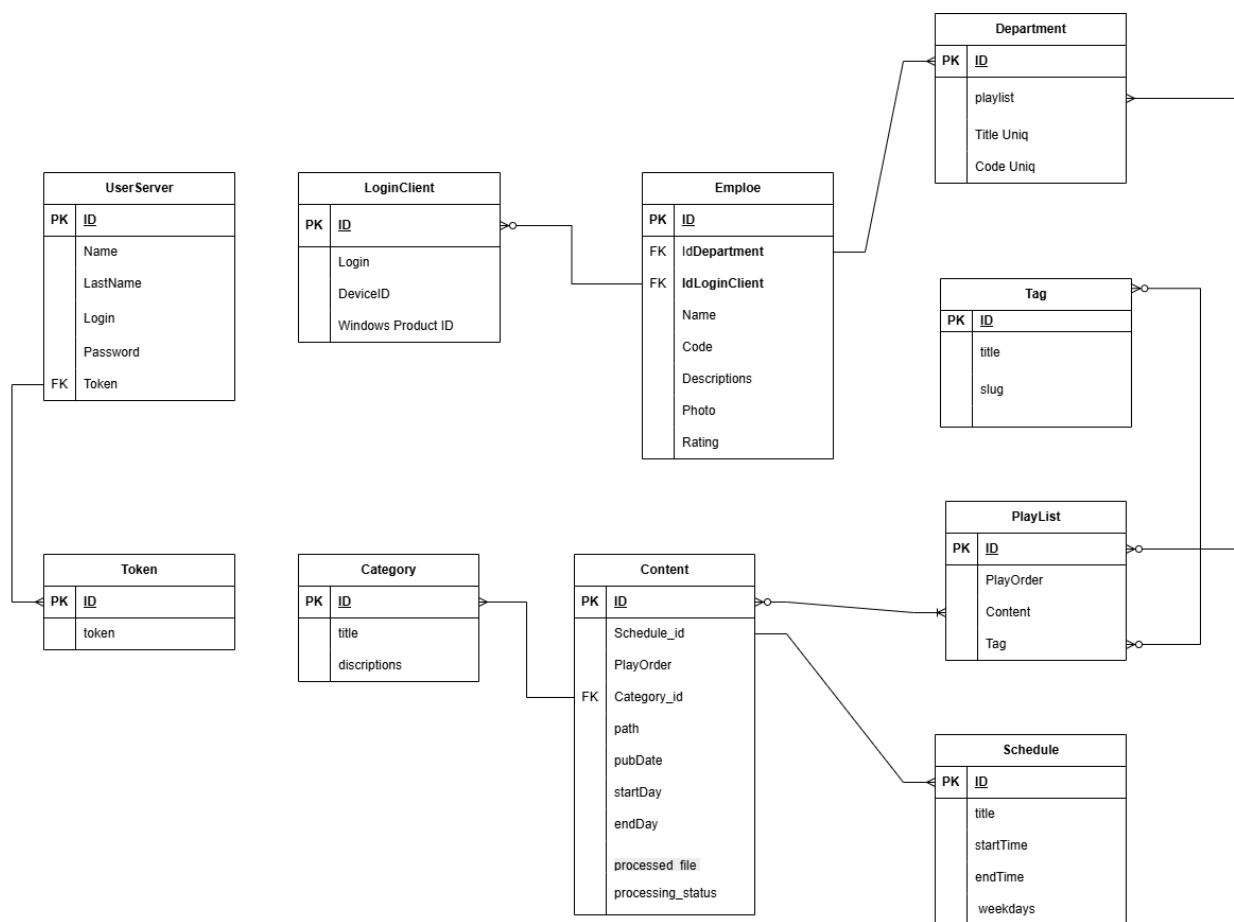


Рисунок 3 – ER-диаграмма серверной части

Описание полей серверной части таблиц базы данных и их связей представлено в словаре данных (Таблица 2).

Таблица 2 – Словарь данных

Наименование	Тип данных	Значение
Category		
id	integer	Первичный ключ, значение автоинкрементируется, начиная с 1
title	varchar(255)	Содержит название категории
descriptions	text	Содержит описание категории
slug	varchar(255)	Содержит транскрипцию наименования категории, служит для построения URL
Users		

id	integer	Первичный ключ, значение автоинкрементируется, начиная с 1
login	varchar(255)	Содержит Login пользователя
password(hash)	text	Содержит пароль в хешированном виде
lastname	varchar(255)	Содержит имя пользователя
firstname	varchar(255)	Содержит фамилию пользователя
token	varchar(255)	Содержит набор символов для авторизации клиентских приложений
Content		
id	integer	Первичный ключ, значение автоинкрементируется, начиная с 1
title	varchar(255)	Содержит наименование видеозаписи
path	varchar(255)	Содержит путь на локальном диске, где хранится видеозапись
playOrder	integer	Порядок воспроизведения
category	varchar(255)	Категория видеозаписи
pubDate	timestamp	Дата загрузки видеозаписи
schedule	integer	(FK) Внешний ключ – хранит Id расписания
startDay	timestamp	День начала трансляции
endDay	timestamp	День окончания трансляции
processed_file	varchar(255)	Содержит путь на локальном диске, где хранится преобразованная видеозапись
processing_status	varchar(255)	Статус конвертации видеоматериала

Продолжение таблицы 2

Department		
id	integer	Первичный ключ, значение автоинкрементируется, начиная с 1
playlist_id	integer	FK) Внешний ключ - хранит Id плейлиста
title	varchar(255)	Содержит наименование отдела
code	varchar(255)	Содержит код отдела

slug	varchar(255)	Содержит транскрипцию наименования, служит для построения URL
Playlist		
id	integer	Первичный ключ, значение автоинкрементируется, начиная с 1
playOrder	integer	Порядок воспроизведения
title	varchar(255)	Содержит наименование плейлиста
tag	varchar(255)	Содержит тег плейлиста
slug	varchar(255)	Содержит транскрипцию наименования, служит для построения URL
Schedule		
id	integer	Первичный ключ, значение автоинкрементируется, начиная с 1
title	varchar(255)	Содержит наименование расписания
startTime	timestamp	Время начала трансляции
endTime	timestamp	Время окончания трансляции
weekdays	array	Содержит дни недели трансляции
Employee		
id	integer	Первичный ключ, значение автоинкрементируется, начиная с 1
name	varchar(255)	Имя сотрудника
lastName	varchar(255)	Фамилия сотрудника
descriptions	integer	Описание сотрудника
department	varchar(255)	FK) Внешний ключ – хранит Id отдела
photo	varchar(255)	Фото сотрудника

Продолжение таблицы 2

login	integer	(FK) Внешний ключ – хранит Id локальной учетной записи
rating	timestamp	Рейтинг сотрудника
endDay	timestamp	День окончания трансляции

processed_file	varchar(255)	Содержит путь на локальном диске, где хранится преобразованная видеозапись
processing_status	varchar(255)	Статус конвертации видеоматериала
Tag		
id	integer	Первичный ключ, значение автоинкрементируется, начиная с 1
title	varchar(255)	Содержит название тега
slug	varchar(255)	Содержит транскрипцию наименования, служит для построения URL
Token		
id	integer	Первичный ключ, значение автоинкрементируется, начиная с 1
token	varchar(255)	Содержит токен авторизации клиентского приложения
LoginClient		
id	integer	Первичный ключ, значение автоинкрементируется, начиная с 1
loginMashine	varchar(255)	Содержит логин учетной записи Windows
ipaddress	varchar(255)	Содержит локальный IP адрес устройства
deviceId	varchar(255)	Содержит токен авторизации клиентского приложения

Диаграмма отношений сущностей, описывающая таблицы БД клиентской части приведена на рисунке 4

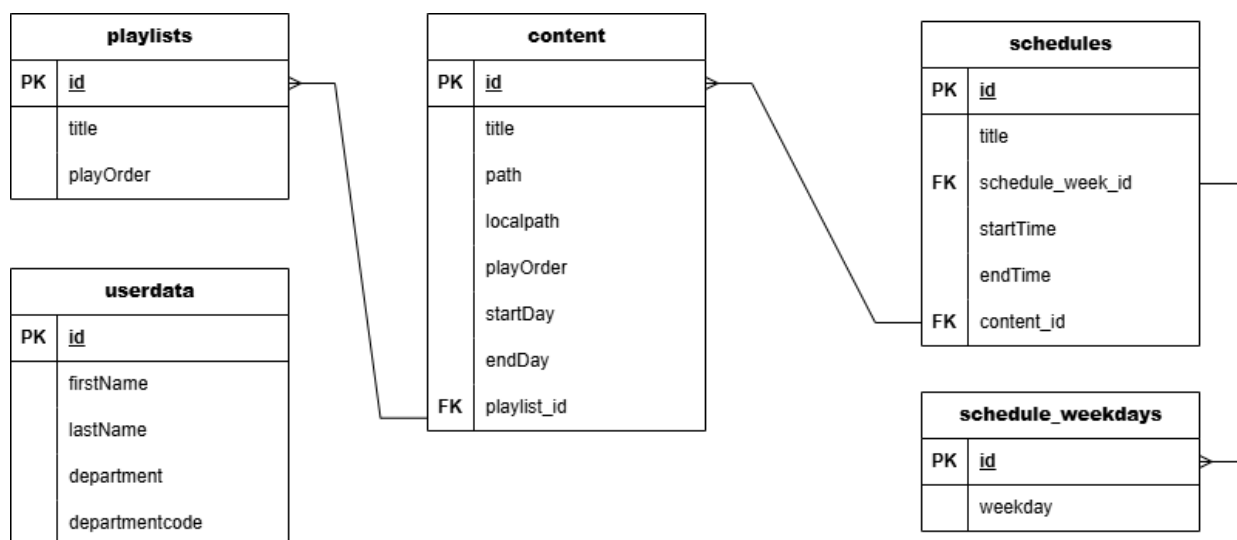


Рисунок 4 – ER-диаграмма клиентской части

Описание полей клиентской части таблиц базы данных и их связей представлено в словаре данных (Таблица 3).

Таблица 3– Словарь данных

Наименование	Тип данных	Значение
playlist		
id	integer	Первичный ключ, значение автоинкрементируется, начиная с 1
title	varchar(255)	Содержит название плейлиста
content		
id	integer	Первичный ключ, значение автоинкрементируется, начиная с 1
title	varchar(255)	Содержит наименование контента
path	varchar(255)	Содержит ссылку для скачивания
localpath	varchar(255)	Содержит путь сохранения файла
playOrder	integer	Порядок воспроизведения
startDay	timestamp	День начала трансляции
endDay	timestamp	День окончания трансляции
playlist_id	integer	Внешний ключ - хранит Id плейлиста

Продолжение таблицы 3

schedules		
id	integer	Первичный ключ, значение автоинкрементируется, начиная с 1
title	varchar(255)	Содержит наименование расписания
content_id	varchar(255)	Внешний ключ - хранит Id контента
Schedules_week_id	varchar(255)	Внешний ключ - хранит Id расписание дней недели
startTime	timestamp	Время начала трансляции
endTime	timestamp	Время окончания трансляции
schedules_weekdays		
id	integer	Первичный ключ, значение автоинкрементируется, начиная с 1
weekday	integer	Содержит день недели (от 0 до 6)
userdata		
id	integer	Первичный ключ, значение автоинкрементируется, начиная с 1
firstname	varchar(255)	Содержит имя сотрудника
lastname	varchar(255)	Содержит фамилию сотрудника
department	varchar(255)	Отдел
departmentcode	timestamp	Код отдела

3.3 Описание сервисов и окружения приложения

Проект серверной части реализован в IDE PyCharm и включает в себя следующие основные компоненты:

В приложении А представлены конфигурационные файлы Dockerfile и docker-compose.yml,

– Dockerfile - конфигурационный файл, в котором описаны инструкции, которые будут применены при сборке Docker-образа и запуске контейнера,

- `docker-compose.yml` - конфигурационный файл в YAML-формате, описывающий логику запуска и взаимодействия контейнеров между собой,
- `manage.py` - является ссылкой на скрипт `django-admin`, с уже предустановленными переменными окружения, указывающими на проект, как для чтения настроек, так и для управления,
 - `settings.py` - настройки проекта,
 - `urls.py` - содержит маршруты URL,
 - `wsgi.py` - файл необходим для подключения к wsgi-серверу, он содержит стандартный интерфейс для запуска приложения,
- директория `models` содержит следующие классы моделей:
 - а) `Category` - описание модели: «Категория контента»;
 - б) `Content` - описание модели: «Контент»;
 - в) `Department` - описание модели: «Отдел»;
 - г) `Employee` - описание модели: «Сотрудник»;
 - д) `LoginClient` - описание модели: «Локальная учетная запись»;
 - е) `Playlist` - описание модели: «Плейлист»;
 - ж) `ScheduleContent` - описание модели: «Расписание»;
 - з) `Tag` - описание модели: «Тег»;
 - и) `Token` - описание модели: «Токен»;
- директория `templates` содержит html файлы, которые отвечают за формирование внешнего вида приложения,
- директория `templatetag` содержит функции, которые отвечают за выборку данных для приложения:
 - а) `show_categories` - выборка из базы данных всех доступных категорий контента;
 - б) `show_employees` - выборка из базы данных сотрудников с наибольшим рейтингом;
 - в) `show_menu` - выборка из базы всех отделов;
 - г) `show_tags` - выборка из базы всех доступных тегов;

– классы View отвечающие за логику приложения, получение данных из моделей и отправку их в шаблоны для отображения на веб-странице:

а) Home - главная страница приложения отображает список плейлистов;

б) Search - поиск плейлистов по названию;

в) GetCategory - отображает список контента конкретной категории;

г) UploadContent - загружает новый контент и привязывает загруженное видео к конкретному плейлисту;

д) AddCategory - добавление новой категории;

е) AddDepartment - добавление нового отдела;

ж) AddPlaylist - создание нового плейлиста;

з) PlaylistByTag – отображение плейлистов с привязанным тегом;

и) GetPlaylist – просмотр контента конкретного плейлиста;

к) GetDepartment - просмотр информации о сотрудниках отдела;

– process_video - функция отвечающая за конвертацию видео к единому формату mp4,

– Классы API отвечающие за передачу данных в формате JSON:

а) DepartmentPlaylistViewSet - передача структурированных данных по отделу с плейлистами, контентом и расписанием;

б) GetDepartmentViewSet - передача всех доступных отделов;

Проект клиентской части реализован в IDE QT Creator и включает в себя следующие основные компоненты:

Приложение заставка:

– HRScreensaver.pro - конфигурационный файл, который содержит информацию необходимую qmake для создания приложения,

– HrScreensaver - класс отвечающий за воспроизведение видеоконтента и установку соединения с сервисом загрузки,

– MainWindow – главное окно настроек приложения.

Сервис загрузки контента:

- Service - представляет собой сервис загрузки контента, который формирует список актуального контента и передает его заставке,

Библиотека CommonModels включает в себя классы для работы с моделью пользовательских данных, сетевым взаимодействием с сервером и работой с базой данных:

- DatabaseManager – класс, отвечающий за работу с локальной базой данных SQLite,

- NetworkManager – данный класс реализует установку соединения с сервером по протоколу http,

- UserDataModel – является моделью пользовательских данных, такие как имя, фамилия, отдел и др.

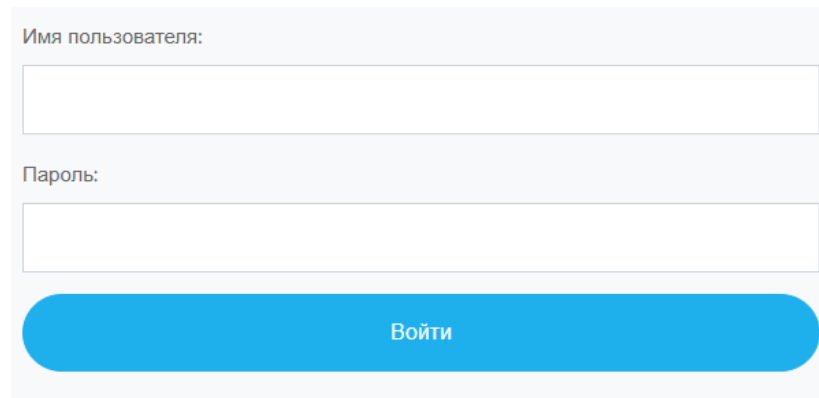
4 Тестирование

Для проверки работоспособности серверной и клиенткой части приложения было проведено функциональное ручное тестирование.

4.1 Тестирование авторизации пользователя в системе

При первом открытии сайта система направляет пользователя на страницу авторизации. Учетная запись администратора создается в системе автоматически, после чего администратор имеет возможность добавлять других пользователей и назначать им соответствующие права доступа. (рисунки 5-6).

Авторизация



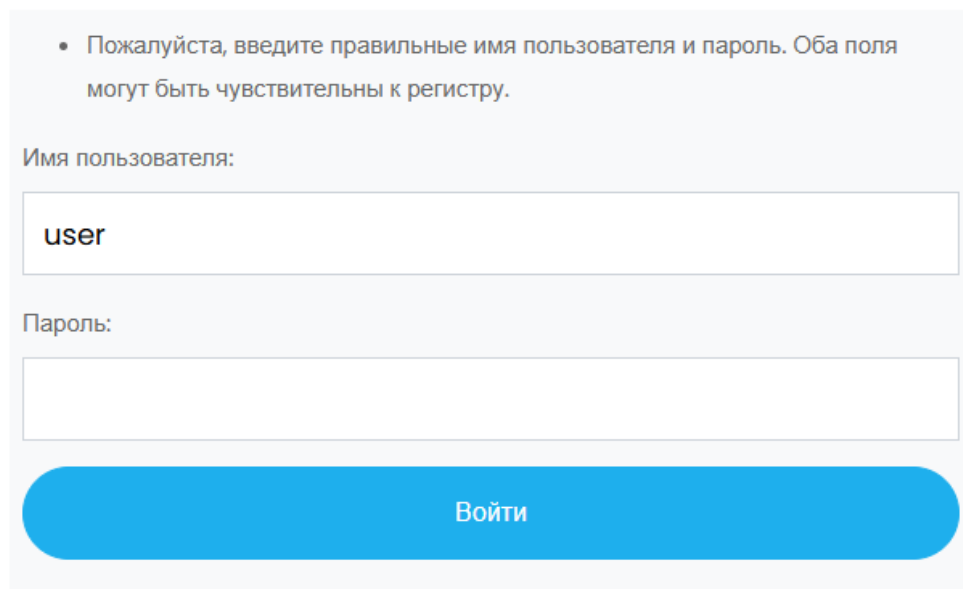
Имя пользователя:

Пароль:

Войти

Рисунок 5 – Форма авторизации

Авторизация



• Пожалуйста, введите правильные имя пользователя и пароль. Оба поля могут быть чувствительны к регистру.

Имя пользователя:

Пароль:

Войти

Рисунок 6 – Ошибка о неверно введенных данных

При успешной авторизации пользователя перенаправляет на главную страницу со списком плейлистов (рисунок 7).

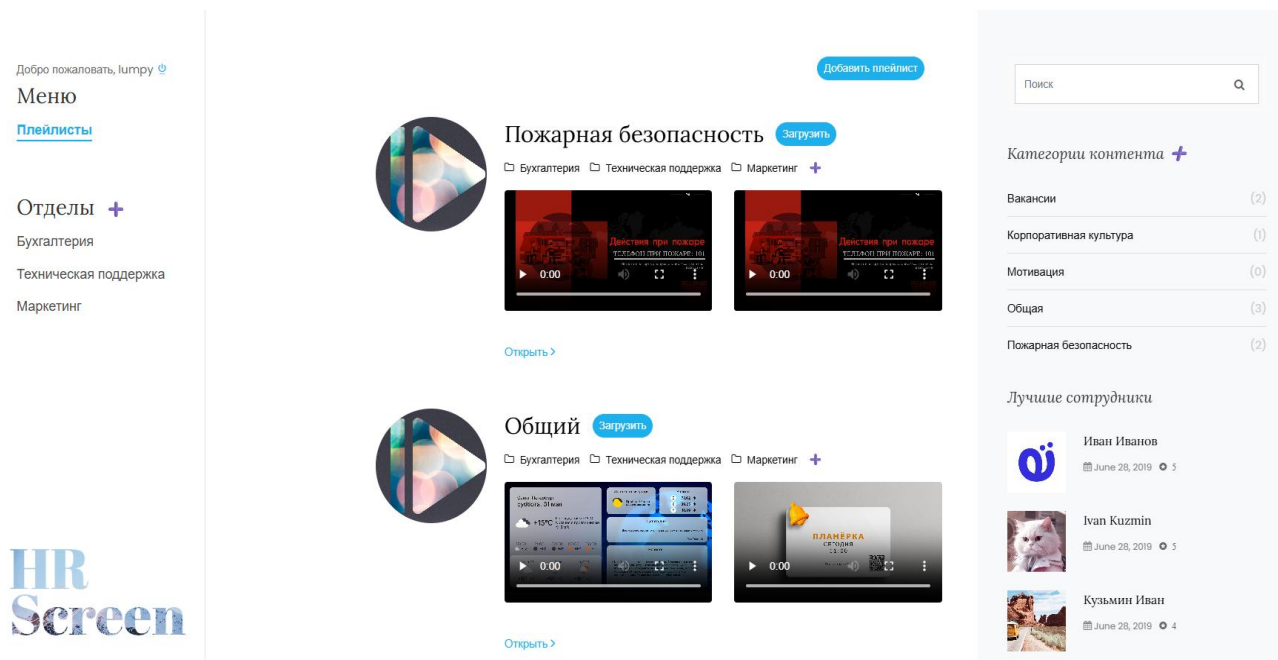


Рисунок 7– Главная страница с отображением плейлистов

4.2 Тестирование взаимодействия с плейлистами, назначение плейлистов на отделы и добавление расписания трансляции

С помощью кнопки «Добавить плейлист» можно добавить новые плейлисты. Тестирование добавления плейлистов и разделение их между отделами представлено на рисунках 8-19.

Добавить плейлист

Название:

Тест

Добавить

Рисунок 8– Страница добавления плейлиста

Плейлист: Тест

Добавить отдел

В плейлисте еще нет видеозаписей!

Загрузить

Рисунок 9– Страница назначения плейлиста на отделы и загрузки контента

Загрузка видео в плейлист: Тест

Название:

Файл:

Выберите файл

Файл не выбран

Категория:

Выберите категорию

▼

Расписание трансляции:

Выберите расписание

▼

Дата начала:

ДД.ММ.ГГГГ

Дата окончания:

ДД.ММ.ГГГГ

Загрузить

Рисунок 10– Страница загрузки контента в плейлист

Загрузка видео в плейлист: Тест

- Допустимы только файлы форматов MP4 и AVI.

Название:

hr

- Допустимы только файлы форматов MP4 и AVI.

Файл:

Выберите файл

Файл не выбран

Категория:

Вакансии

Расписание трансляции:

Весь день

Дата начала:

25.06.2025

Дата окончания:

31.07.2025

Загрузить

Рисунок 11– Выбор не подходящих файлов для загрузки

Загрузка видео в плейлист: Тест

Название:	photo_2024-07-09_20-25-24_wSGdM5T
Файл:	Выберите файл photo_2024-07-09_20-25-24_wSGdM5T.m4v
Категория:	Корпоративная культура
Расписание трансляции:	Весь день
Дата начала:	23.06.2025
Дата окончания:	31.07.2025
Загрузить	

Рисунок 12 – Попытка загрузки видеофайла подходящего для загрузки

После успешной загрузки видеофайла на сервер, происходит процесс конвертации видеозаписи к единому формату отображения, т.к. процесс конвертации может занимать длительное время, вместо видеофайла отображается статичное изображение (рисунок 13).

Плейлист: Тест



Рисунок 13 – Процесс подготовки видеофайла к отображению

Плейлист: Вакансии

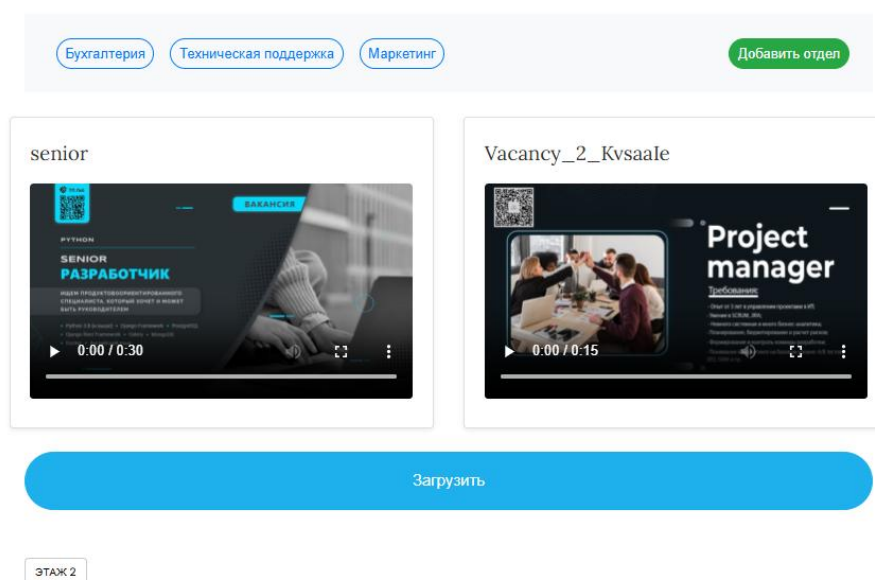


Рисунок 14 – Отображение готовых видеофайлов

После добавления видеофайлов в плейлист необходимо назначить отделы, для которых данный плейлист предназначен. (рисунок 15).

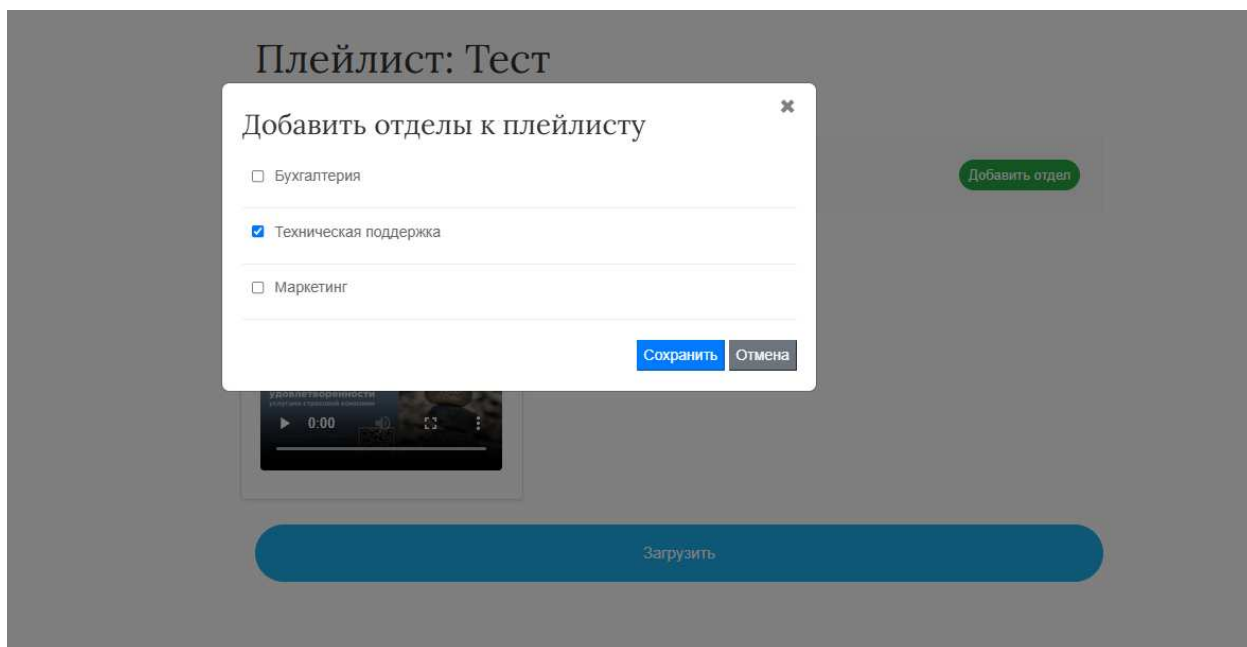


Рисунок 15 – Назначение отделов на плейлист.

Также, для удобства использования, данный функционал продублирован на главной странице управления плейлистами в виде кнопки «+» (рисунок 16).

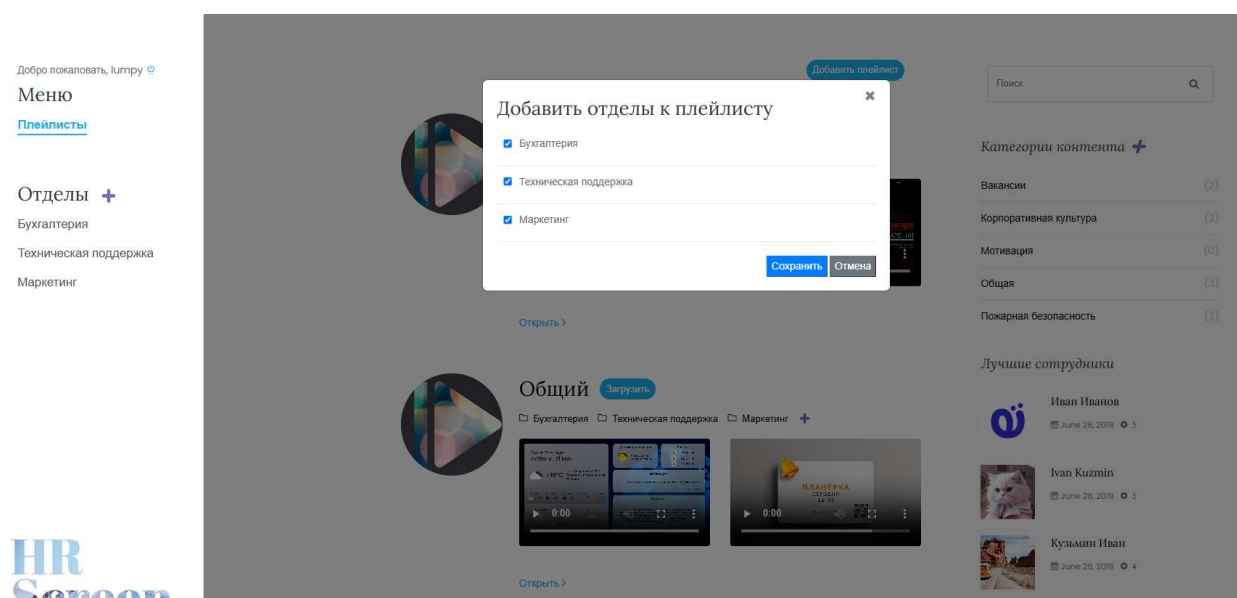


Рисунок 16 – Назначение отделов на плейлист на главной странице.

Добавить Расписание

Title:	Утро
StartTime:	10 Сейчас 🕒
Внимание: Ваше локальное время опережает время сервера на 3 часа.	
EndTime:	12 Сейчас 🕒
Внимание: Ваше локальное время опережает время сервера на 3 часа.	
Weekdays:	☒ Monday ☐ Tuesday ☒ Wednesday ☐ Thursday ☒ Friday ☐ Saturday ☐ Sunday

СОХРАНИТЬ

Сохранить и добавить другой объект

Сохранить и продолжить редактирование

Рисунок 17 – Добавление нового расписания для трансляции с 10:00 до 12:00 каждый понедельник, среду и пятницу.

После чего при загрузке файлов будет доступно новое расписание трансляции (рисунок 18-19).

Загрузка видео в плейлист: Тест

Название:

photo_2024-07-09_20-25-24_wSGdM5T

Файл:

Выберите файл photo_2024-07-09_20-25-24_wSGdM5T.m4v

Категория:

Корпоративная культура

Расписание трансляции:

Выберите расписание

Выберите расписание

Весь день

Утро

Дата окончания:

ДД.ММ.ГГГГ

Загрузить

Рисунок 18 – Выбор нового расписания трансляции для видеофайла.

Загрузка видео в плейлист: Тест

Название:

photo_2024-07-09_20-25-24_wSGdM5T

Файл:

Выберите файл photo_2024-07-09_20-25-24_wSGdM5T.m4v

Категория:

Корпоративная культура

Расписание трансляции:

Выберите расписание

Дата начала:

06.05.2025

Май 2025

Пн	Вт	Ср	Чт	Пт	Сб	Вс
28	29	30	1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30	31	1
2	3	4	5	6	7	8

Удалить Сегодня

Загрузить

Рисунок 19 – Выбор сроков ротации контента по календарным дням.

4.3 Тестирование API

В связи с изменением настроек на главной странице плейлистов, все изменения учитываются в клиентском API, тем самым при последующем обращении, клиенты получают обновленные данные о видеофайлах и их сроках ротации. Для идентификации устройства, используется токен авторизации, который передается в заголовочных файлах при обращении к API. Тестирование API представлено на рисунках 20 – 24.

```
Администратор: C:\WINDOW
Microsoft Windows [Version 10.0.26100.4061]
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

C:\Users\super>curl http://10.90.0.1:8000/api/department/?code=01
{"detail": "Учетные данные не были предоставлены."}
C:\Users\super>
```

Рисунок 20 – Обращение к API без предоставления токена авторизации с помощью утилиты CURL

```
Администратор: C:\WINDOW
C:\Users\super>curl -H "Authorization: Token 230875e6-d857-4853-a6b0-0aac83fa9142" http://10.90.0.1:8000/api/department/?code=01
[{"id":1,"title":"Техническая поддержка","code":"01","playLists":[{"id":1,"name":"Общий","playOrder":1,"content":[{"id":49,"title":"Content - Санкт-Петербург_1 fsTtWw","path":"http://10.90.0.1:8000/media/processed/2025/06/01/converted_49.mp4","playOrder":49,"startDay":"2025-05-15","endDay":"2025-10-03","schedule":{"id":1,"startTime":"00:00:00","endTime":"23:59:59","weekdays":["0","1","2","3","4","5","6"]}},{"id":50,"title":"Планёрка","path":"http://10.90.0.1:8000/media/processed/2025/06/02/converted_50.mp4","playOrder":50,"startDay":"2025-05-09","endDay":"2025-10-05","schedule":{"id":1,"startTime":"00:00:00","endTime":"23:59:59","weekdays":["0","1","2","3","4","5","6"]}},{"id":7,"name":"9 Мая","playOrder":7,"content":[{"id":51,"title":"5145996_504C09N","path":"http://10.90.0.1:8000/media/processed/2025/06/02/converted_51.mp4","playOrder":51,"startDay":"2025-06-17","endDay":"2025-08-29","schedule":{"id":1,"startTime":"00:00:00","endTime":"23:59:59","weekdays":["0","1","2","3","4","5","6"]}},{"id":22,"name":"Вакансии","playOrder":22,"content":[{"id":52,"title":"senior","path":"http://10.90.0.1:8000/media/processed/2025/06/05/converted_52.mp4","playOrder":52,"startDay":"2025-05-16","endDay":"2025-06-29","schedule":{"id":1,"startTime":"00:00:00","endTime":"23:59:59","weekdays":["0","1","2","3","4","5","6"]}},{"id":53,"title":"Vacancy_2_KvsaaIe","path":"http://10.90.0.1:8000/media/processed/2025/06/05/converted_53.mp4","playOrder":53,"startDay":"2025-05-23","endDay":"2025-08-29","schedule":{"id":1,"startTime":"00:00:00","endTime":"23:59:59","weekdays":["0","1","2","3","4","5","6"]}},{"id":23,"name":"Пожарная безопасность","playOrder":23,"content":[{"id":47,"title":"Fire_safety_page_7","path":"http://10.90.0.1:8000/media/processed/2025/06/01/converted_47.mp4","playOrder":47,"startDay":"2025-05-14","endDay":"2025-11-08","schedule":{"id":1,"startTime":"00:00:00","endTime":"23:59:59","weekdays":["0","1","2","3","4","5","6"]}},{"id":48,"title":"Fire safety page 7","path":"http://10.90.0.1:8000/media/processed/2025/06/01/converted_48.mp4","playOrder":48,"startDay":"2025-03-07","endDay":"2025-08-31","schedule":{"id":1,"startTime":"00:00:00","endTime":"23:59:59","weekdays":["0","1","2","3","4","5","6"]}},{"id":24,"name":"Корпоративная культура","playOrder":24,"content":[{"id":54,"title":"photo_2024-07-09_20-25-24_wSgdM5T","path":"http://10.90.0.1:8000/media/processed/2025/06/05/converted_54.mp4","playOrder":54,"startDay":"2025-06-14","endDay":"2025-08-22","schedule":{"id":1,"startTime":"00:00:00","endTime":"23:59:59","weekdays":["0","1","2","3","4","5","6"]}},{"id":25,"name":"Тест","playOrder":25,"content":[{"id":55,"title":"photo_2024-07-09_20-25-24_wSgdM5T","path":"http://10.90.0.1:8000/media/processed/2025/06/06/converted_55.mp4","playOrder":55,"startDay":"2025-06-23","endDay":"2025-07-31","schedule":{"id":1,"startTime":"00:00:00","endTime":"23:59:59","weekdays":["0","1","2","3","4","5","6"]}}]]]}]
```

Рисунок 21 – Обращение к API с предоставлением токена авторизации с помощью утилиты CURL

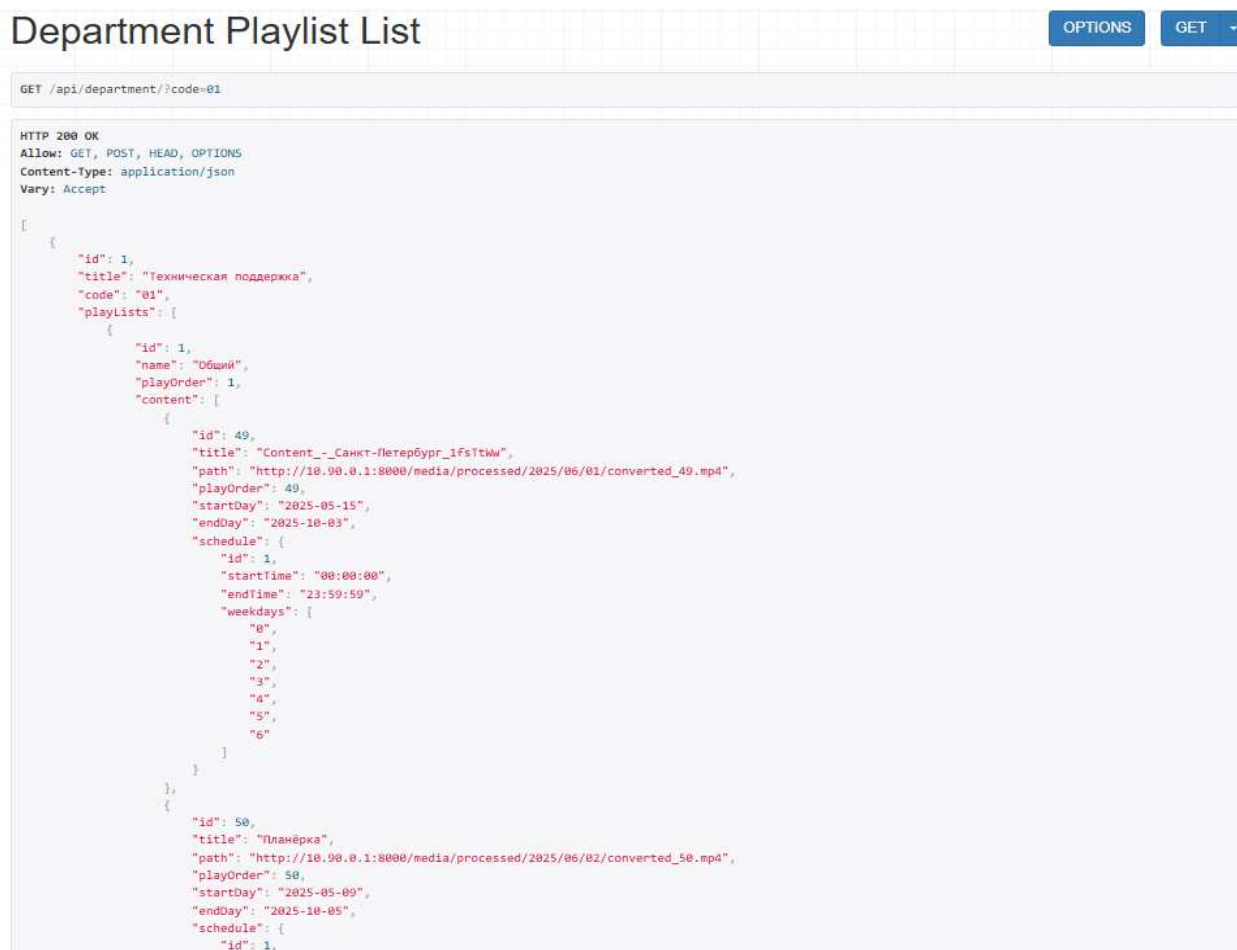


Рисунок 22 – Обращение к API с предоставлением токена авторизации с помощью веб интерфейса

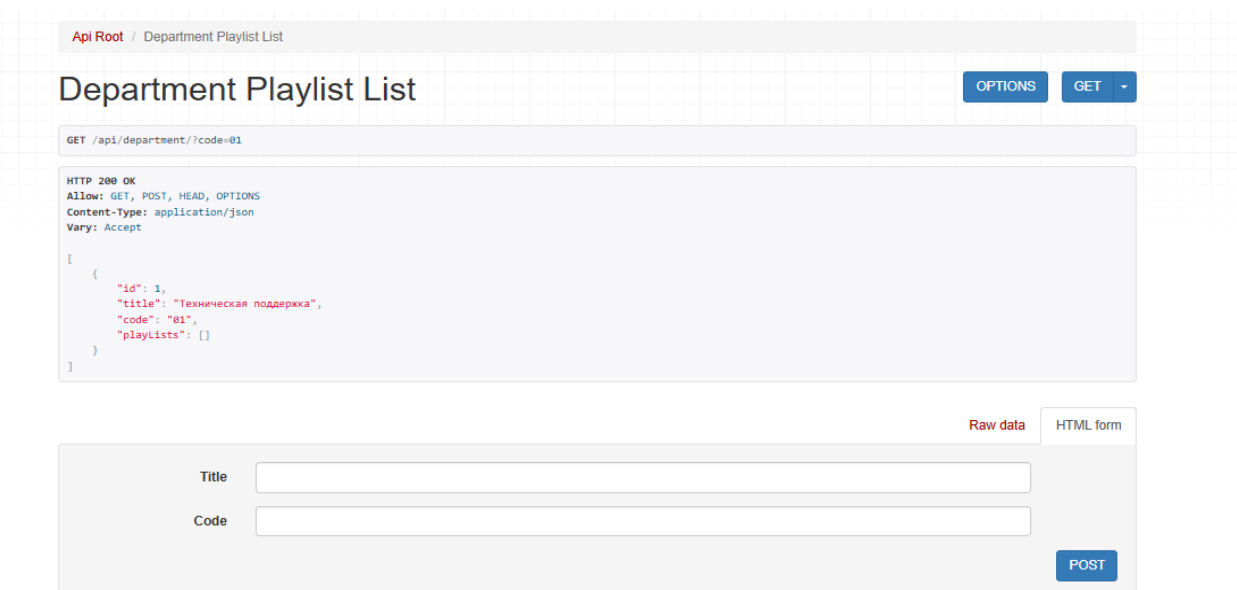


Рисунок 23 – Снятие с ротации всех плейлистов в отделе: «Техническая поддержка»

```

HTTP 200 OK
Allow: GET, POST, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

[
  {
    "id": 1,
    "title": "Техническая поддержка",
    "code": "01",
    "playlists": [
      {
        "id": 23,
        "name": "Пожарная безопасность",
        "playOrder": 23,
        "content": [
          {
            "id": 47,
            "title": "Fire_safety_page_7",
            "path": "http://10.90.0.1:8000/media/processed/2025/06/01/converted_47.mp4",
            "playOrder": 47,
            "startDay": "2025-05-14",
            "endDay": "2025-11-08",
            "schedule": {
              "id": 1,
              "startTime": "00:00:00",
              "endTime": "23:59:59",
              "weekdays": [
                "0",
                "1",
                "2",
                "3",
                "4",
                "5",
                "6"
              ]
            }
          },
          {
            "id": 48,
            "title": "Fire_safety_page_7",
            "path": "http://10.90.0.1:8000/media/processed/2025/06/01/converted_48.mp4",
            "playOrder": 48,
            "startDay": "2025-03-07",
            "endDay": "2025-08-31",
            "schedule": {
              "id": 1,
              "startTime": "00:00:00",
              "endTime": "23:59:59",
              "weekdays": [
                "0",
                "1",
                "2",
                "3",
                "4",
                "5",
                "6"
              ]
            }
          }
        ]
      }
    ]
  }
]

```

Рисунок 24 – Назначение плейлиста «Пожарная безопасность» в отделе: «Техническая поддержка»

4.3 Тестирование клиентского приложения «Заставка»

При первоначальной настройке приложения необходимо указать ФИО сотрудника и выбрать отдел. Тестирование приложения представлено на рисунках 25 – 30.

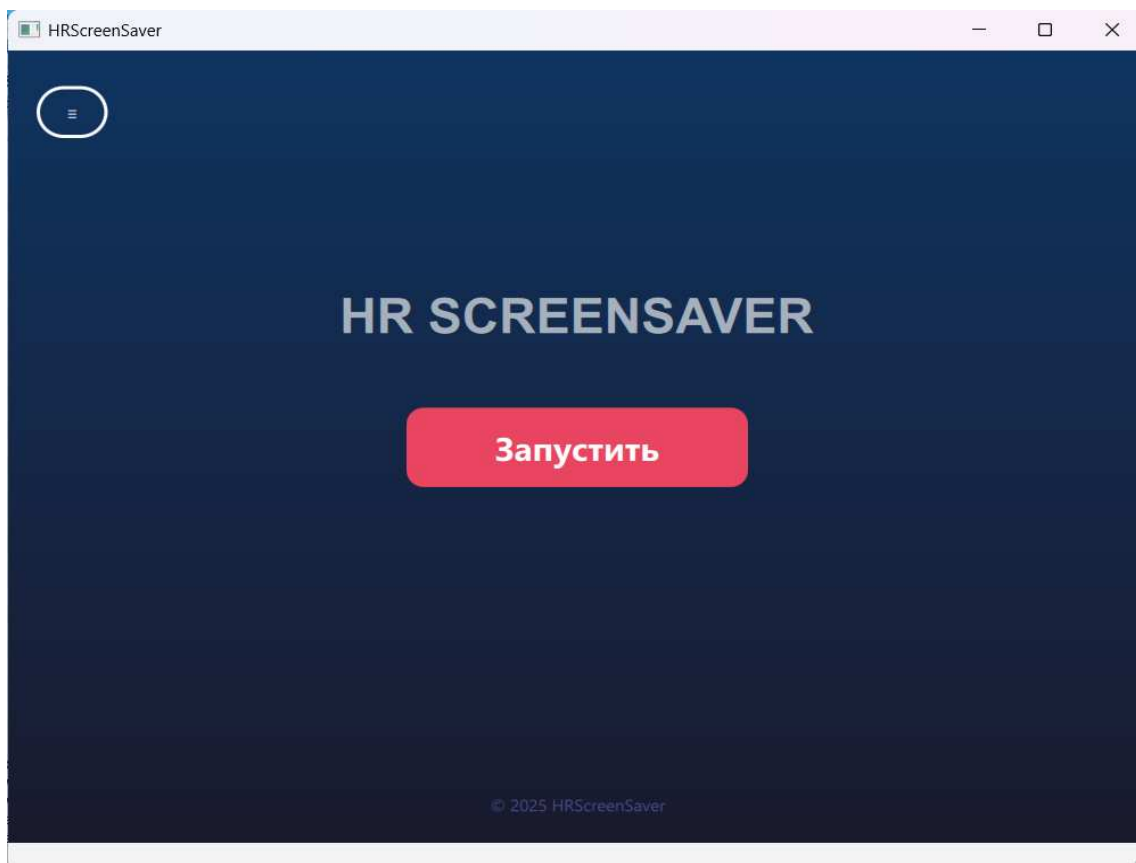


Рисунок 25 – Главное окно приложения

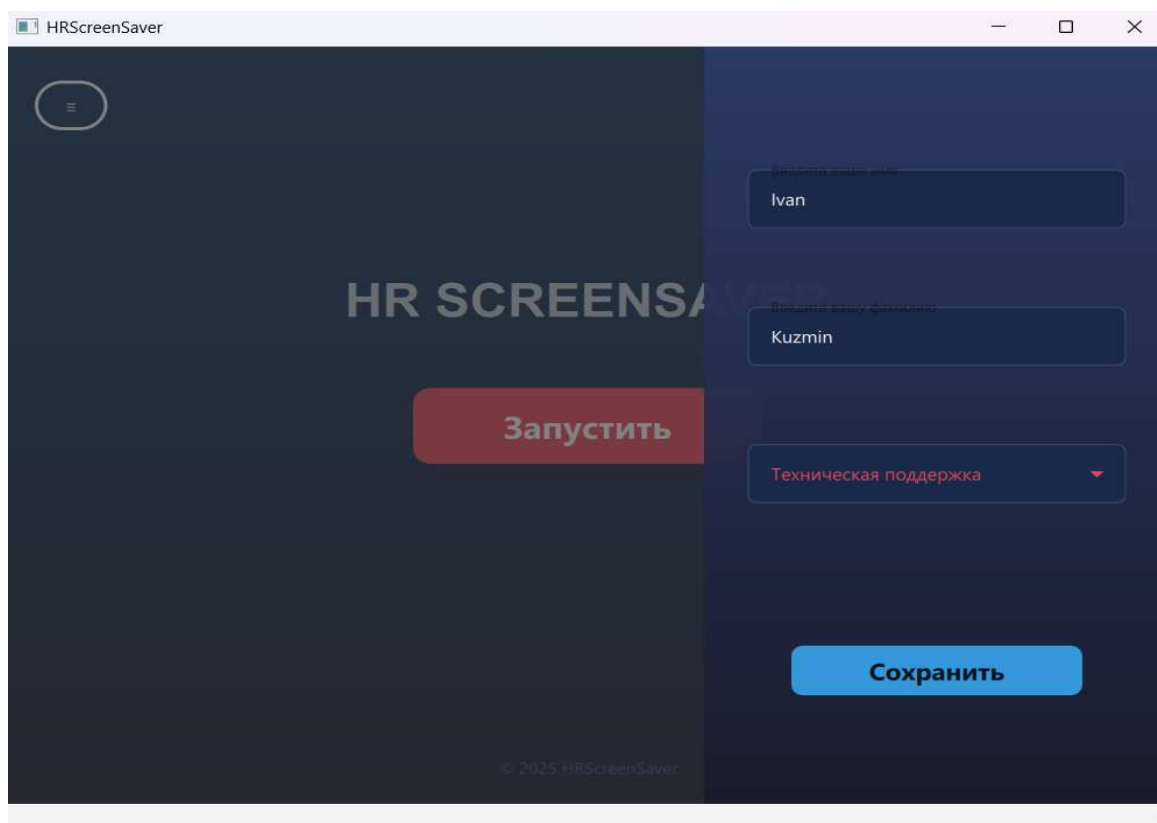


Рисунок 26 – Поля ввода ФИО сотрудника и выбор отдела.



Рисунок 27 – Запуск приложения в режиме заставки

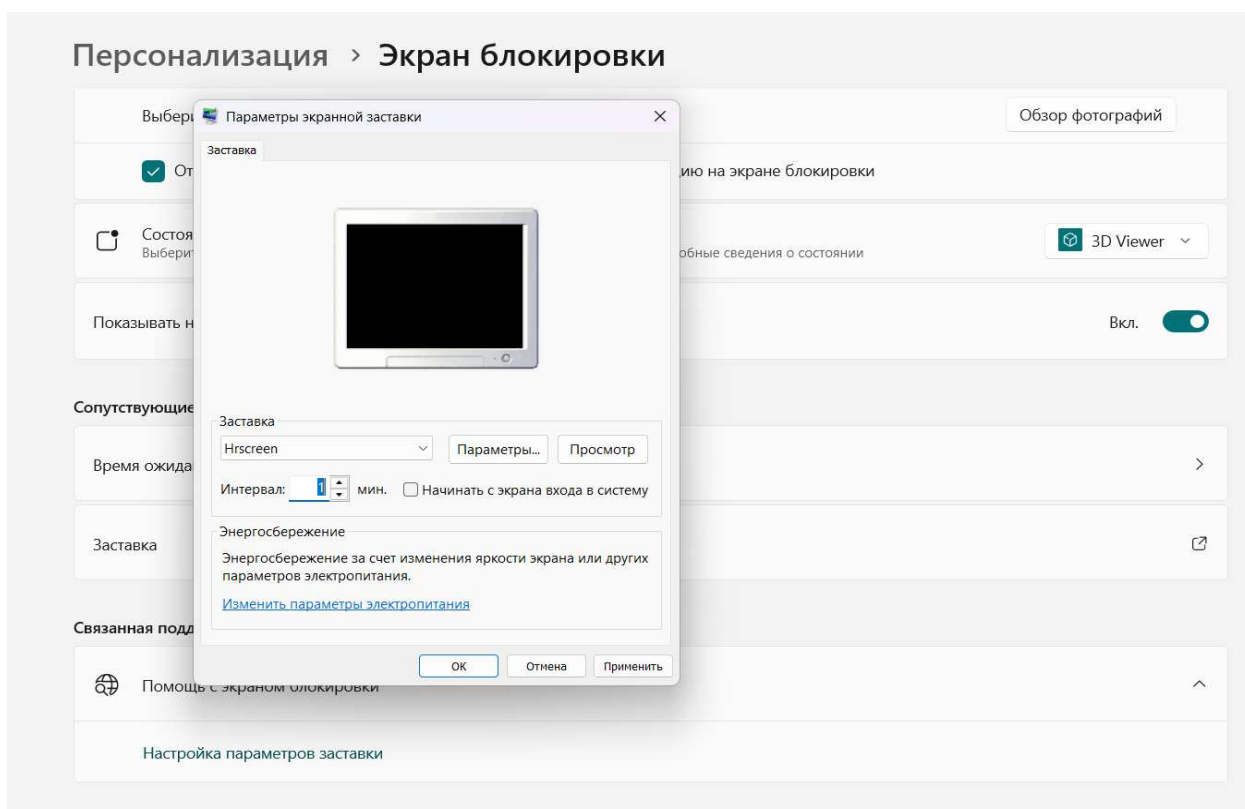


Рисунок 28 – Настройка приложения в качестве штатной заставки Windows

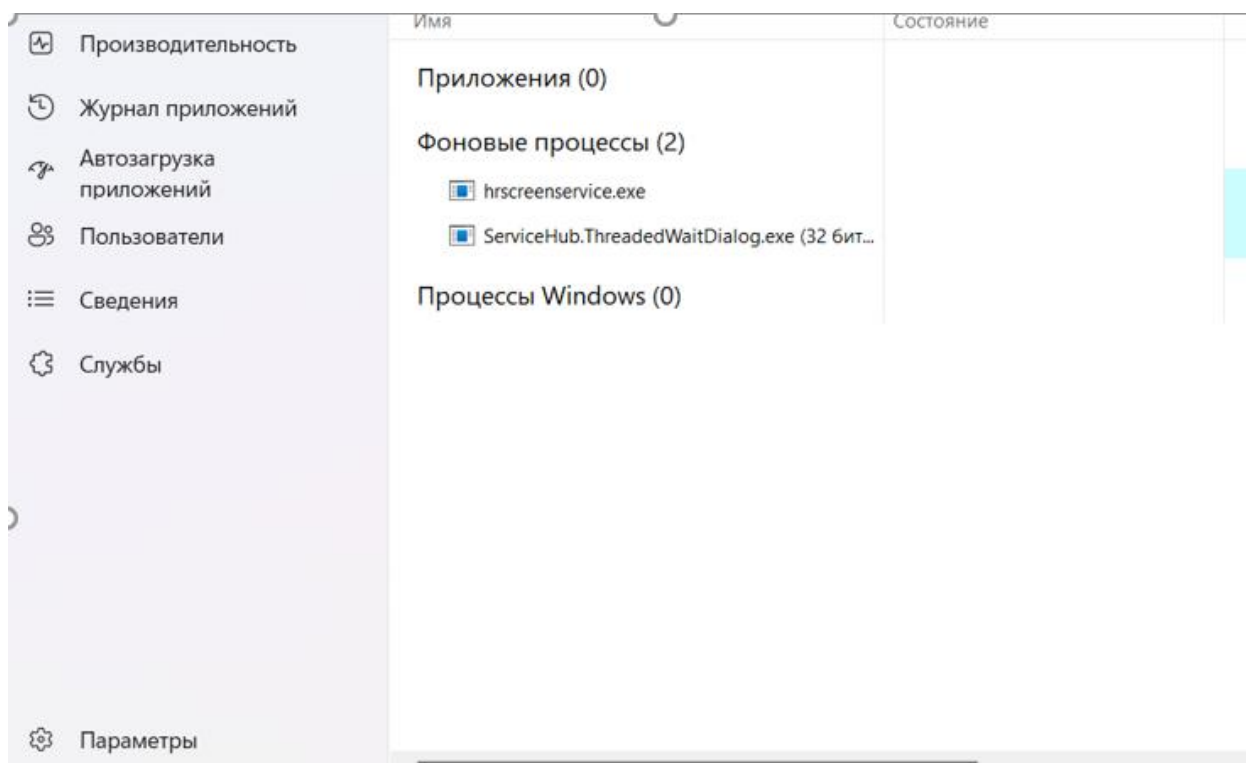


Рисунок 29 – Работа фонового процесса для получения данных с сервера

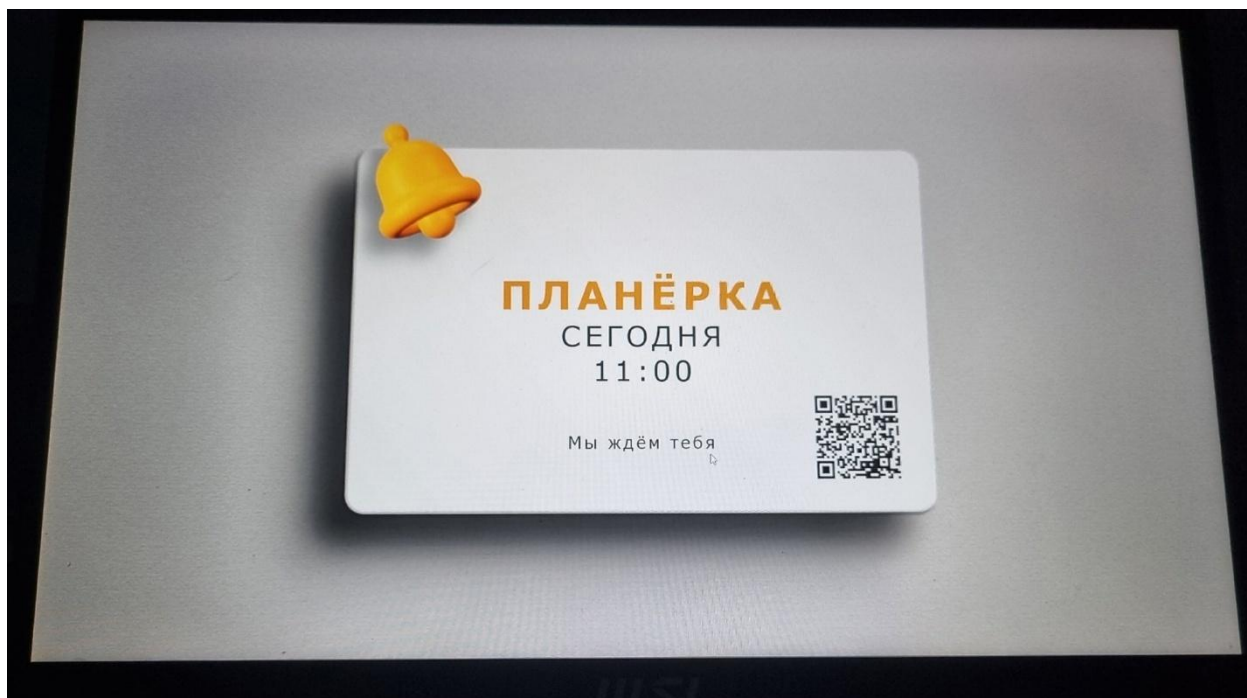


Рисунок 30 – Запуск приложения в качестве штатной заставки Windows

ЗАКЛЮЧЕНИЕ

В процессе выполнения выпускной квалификационной работы была достигнута поставленная цель — разработка современной платформы для корпоративного телевидения, направленной на оптимизацию внутренней коммуникации и управление видеоконтентом в организации. Система обеспечивает централизованное управление контентом, настройку персонализированных трансляций по отделам, гибкое планирование ротации видеоматериалов по времени и дням недели, а также реализована функция экранной заставки.

Подводя итоги выполнения выпускной квалификационной работы, можно сделать следующие выводы:

1. Проведен детальный анализ предметной области, включающий исследование актуальности проблемы, сравнительный анализ существующих аналогов и выявление их недостатков. На основе данного анализа были сформулированы функциональные и нефункциональные требования к разрабатываемой системе.

2. Реализовано приложение, соответствующее всем установленным требованиям. Разработанная платформа предоставляет полный комплекс инструментов для эффективного управления внутренними корпоративными коммуникациями через видеоконтент.

В дальнейшем планируется расширение функционала созданной системы, включающее следующие улучшения:

1. Внедрение системы детальной статистики по просмотрам контента и уровню вовлеченности сотрудников.

2. Добавление функционала интеграции опросов, викторин и механизмов обратной связи в рамках транслируемого контента.

3. Разработка рекомендательной системы контента, основанной на роли, отделе и активности сотрудников.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Осовицкая, Н. (2020). HR DIGITAL. Практики лучших работодателей. Санкт-Петербург: Издательство "Питер".
2. Вовлечённость, или когда работнику не все равно URL: <https://salttraining.ru/articles/vovlechjonnost-ili-kogda-rabotniku-ne-vse-ravno/> (дата обращения: 11.05.2025).
3. Вовлеченность и удовлетворенность сотрудников: что и зачем необходимо измерять? URL: <https://ht-lab.ru/blog/vovlechnennost-i-udovletvorennost-sotrudnikov-chto-i-zachem-neobkhodimo-izmeryat/> (дата обращения: 11.05.2025).
4. Результаты опроса: эмоциональное состояние сотрудников в 2022 году URL: <https://ht-lab.ru/blog/vliyanie-ehmocij-na-rabotu/> (дата обращения: 09.05.2025).
5. Сколько стоят неэффективные бизнес-коммуникации: исследование SKOLKOVO URL: <https://www.skolkovo.ru/news/skolko-stoyat-neeffectivnye-biznes-kommunikacii-issledovanie-skolkovo/> (дата обращения: 08.05.2025).
6. Треть работодателей тратят на адаптацию одного нового сотрудника до 100 тысяч рублей URL: <https://www.novostiitkanala.ru/news/detail.php?ID=177015> (дата обращения: 08.05.2025).
7. Можно ли с помощью корпоративного ТВ решать задачу сплочения распределенной команды URL: <https://vc.ru/hr/1192005-mozhno-li-s-pomoshyu-korporativnogo-tv-reshat-zadachu-splocheniya-raspredelennoi-komandy> (дата обращения: 08.05.2025).
8. Функциональные и нефункциональные требования: различия и особенности URL: <https://practicum.yandex.ru/blog/funkcionalnye-i-nefunkcionalnye-trebovaniya/> (дата обращения: 08.05.2025).
9. Зимин, А.Н. (2023). "Django 4: практика создания веб-приложений". СПб.: БХВ. - 800 с.

10. Меле, А. (2025). " Django 5 в примерах:". СПб.: Питер. М.: ДМК Пресс. - 866 с.
11. Мельников, В.П. (2023). "Celery: распределенные задачи в Python". М.: ДМК Пресс. - 310 с.
12. Прохоренок, Н.А. (2023). " Qt 6. Разработка оконных приложений на C++ ". СПб.: БХВ. - 512 с.
13. Мартин, Р. (2022). " Чистая архитектура. Искусство разработки программного обеспечения ". М.: Диалектика. - 352 с.
14. Салахалдин Джуба, Андрей Волков Изучаем PostgreSQL 10. - 2-е изд. - Москва: ДМК Пресс, 2019. - 400 с.
15. Комаров В.И. Путеводитель по базам данных. - М.: ДМК Пресс, 2024. - 520 с.

Конфигурационный файл Dockerfile:

```
FROM ubuntu:22.04

COPY requirements.txt /temp/requirements.txt
COPY hrscreen /hrscreen
WORKDIR /hrscreen
EXPOSE 8000
RUN apt-get update && apt-get install postgresql-client -y
RUN apt install python3-pip -y
RUN apt-get install ffmpeg -y
RUN pip install -r /temp/requirements.txt
```

Конфигурационный файл docker-compose.yml:

```
version: '3.0'
services:
  web-app:
    build:
      context: .
    container_name: hrscreen_django
    ports:
      - "10.90.0.1:8000:8000"
    volumes:
      - ./hrscreen:/hrscreen
    environment:
      - DB_HOST=database
      - DB_NAME=hrscreen
      - DB_USER=hrscreen
      - DB_PASS=hrscreen
    depends_on:
      - database
    command: >
      sh -c "python3 manage.py makemigrations \
        && python3 manage.py migrate \
        && python3 manage.py runserver 0.0.0.0:8000"

  database:
    image: postgres
    container_name: hrscreen_postgres
```

```

ports:
  - "10.90.0.1:5432:5432"
environment:
  - POSTGRES_DB=hrscreen
  - POSTGRES_USER=hrscreen
  - POSTGRES_PASSWORD=hrscreen

redis:
  image: redis
  container_name: hrscreen_redis
  hostname: redis
  ports:
    - "10.90.0.1:16379:6379"

worker:
  build:
    context: .
  hostname: worker
  container_name: hrscreen_celery
  entrypoint: celery
  command: "-A hrscreen worker --loglevel=info"
  volumes:
    - ./hrscreen:/hrscreen
  links:
    - redis
  depends_on:
    - redis

```

Приложение Б

Исходный код клиентской части приложения заставки.

hrscreensaver.h

```

#ifndef HRSCREENSAVER_H
#define HRSCREENSAVER_H

#include <QObject>
#include <QWidget>
#include <QLocalSocket>
#include <QMediaPlayer>
#include <QVideoWidget>
#include <QJsonDocument>
#include <QJsonObject>
#include <QLabel>

```

```

class HrScreensaver:public QVideoWidget {
    Q_OBJECT
public:
    HrScreensaver(QWidget *parent = nullptr);
    ~HrScreensaver();

public slots:
    void requestContent();

private slots:
    void onReadyRead();
    void reconnectToServer();
    void handleMediaStatusChanged(QMediaPlayer::MediaStatus status);
    void handleEmptyPlaybackTimeout();
    void playScreensaver();

private:
    QLocalSocket *socket;
    QMediaPlayer *player;
    QStringList playlist;
    QByteArray jsonBuffer;
    QTimer *emptyPlayTimer;
    bool isPlayingScreensaver = false;
    void playNextVideo();
    void processJsonDocument(const QJsonDocument &doc);
    void mousePressEvent(QMouseEvent *event) override;
    void keyPressEvent(QKeyEvent *event) override;

};

#endif // HRSCREENSAVER_H

```

hrscreensaver.cpp

```

#include "hrscreensaver.h"
#include <QDebug>
#include <QMessageBox>
#include <QTimer>
#include <QFile>
#include <QJsonParseError>
#include <QJsonArray>
#include <QKeyEvent>

```



```

#include <QVBoxLayout>

HrScreensaver::HrScreensaver(QWidget *parent): QVideoWidget(parent) {
    player = new QMediaPlayer(this);
    player->setVideoOutput(this);
    emptyPlayTimer = new QTimer(this);
    emptyPlayTimer->setSingleShot(true);
    setMouseTracking(true);
    setFocusPolicy(Qt::StrongFocus);

    socket = new QLocalSocket(this);

    connect(player, &QMediaPlayer::errorOccurred, [this](QMediaPlayer::Error,
const QString &errorString) {
        qDebug() << "Media Error:" << errorString;
        playNextVideo(); // Переход к следующему видео при ошибке
    });

    connect(player, &QMediaPlayer::mediaStatusChanged,
[this](QMediaPlayer::MediaStatus status) {
        if (status == QMediaPlayer::EndOfMedia) {
            playNextVideo(); // Автопереход при завершении видео
        }
    });

    connect(socket, &QLocalSocket::readyRead, this,
&HrScreensaver::onReadyRead);
    connect(socket, &QLocalSocket::connected, this,
&HrScreensaver::requestContent);
    connect(socket, &QLocalSocket::errorOccurred, this,
&HrScreensaver::reconnectToServer);
    connect(player, &QMediaPlayer::mediaStatusChanged, this,
&HrScreensaver::handleMediaStatusChanged);
    connect(emptyPlayTimer, &QTimer::timeout, this,
&HrScreensaver::handleEmptyPlaybackTimeout);
    connect(emptyPlayTimer, &QTimer::timeout, this,
&HrScreensaver::requestContent);
    reconnectToServer();
}

void HrScreensaver::reconnectToServer() {

```

```

        if (socket->state() != QLocalSocket::ConnectedState) {
            socket->connectToServer("hrscreen_screensaver");
            if (!socket->waitForConnected(2000)) {
                qDebug() << "Connection error. Retrying in 5s...";
                QTimer::singleShot(5000, this,
&HrScreensaver::reconnectToServer);
            } else {
                qDebug() << "Connected to server!";
            }
        }
    }

void HrScreensaver::mousePressEvent(QMouseEvent *event) {
    Q_UNUSED(event);
    qDebug() << "Mouse click detected, closing screensaver";
    this->close();
}

void HrScreensaver::handleMediaStatusChanged(QMediaPlayer::MediaStatus
status) {
    if (status == QMediaPlayer::EndOfMedia) {
        if (player->source() == QUrl("qrc:/qml/qml/HRScreen.mp4")) {
            qDebug() << "Restarting screen saver video";
            player->play();
        }
    }
}

void HrScreensaver::handleEmptyPlaybackTimeout() {
    qDebug() << "1 min of screensaver playback completed";
    if (!playlist.isEmpty()) {
        isPlayingScreensaver = false;
        player->stop();
        playNextVideo();
    }
}

void HrScreensaver::keyPressEvent(QKeyEvent *event) {
    if (event->key() == Qt::Key_Escape) {
        qDebug() << "Escape key pressed, closing screensaver";
        this->close();
    }
}

```

```

    } else {
        QVideoWidget::keyPressEvent(event);
    }
}

void HrScreensaver::playScreensaver() {
    qDebug() << "Starting screen saver playback";
    isPlayingScreensaver = true;

    const QString videoPath = "qrc:/qml/qml/HRSscreen.mp4";
    player->setSource(QUrl(videoPath));
    player->play();

    emptyPlayTimer->start(60*1000); // 1 МИНУТ
}

void HrScreensaver::requestContent() {
    if (socket->state() == QLocalSocket::ConnectedState) {
        QJsonObject request;
        request["command"] = "get_content";
        QJsonDocument doc(request);
        socket->write(doc.toJson());
        socket->flush();
        qDebug() << "Requested content from server";
    }
}

void HrScreensaver::onReadyRead() {
    QByteArray newData = socket->readAll();
    qDebug() << "Received data:" << newData;
    jsonBuffer.append(newData);
    while (!jsonBuffer.isEmpty()) {
        QJsonParseError parseError;
        QJsonDocument doc = QJsonDocument::fromJson(jsonBuffer, &parseError);

        if (parseError.error == QJsonParseError::NoError) {
            processJsonDocument(doc);

            int docSize = doc.toJson(QJsonDocument::Compact).size();
            jsonBuffer = jsonBuffer.mid(docSize);
        }
    }
}

```

```

else if (parseError.error == QJsonParseError::UnterminatedObject) {
    qDebug() << "Partial JSON received, waiting for more data";
    break;
}
else {
    qDebug() << "JSON Error:" << parseError.errorString() << "at
offset" << parseError.offset;

    if (parseError.offset == 0) {
        jsonBuffer.remove(0, 1);
    }
    else {
        jsonBuffer = jsonBuffer.mid(parseError.offset);
    }
}

while (!jsonBuffer.isEmpty() && (jsonBuffer[0] == '\n' ||
jsonBuffer[0] == '\r' || jsonBuffer[0] == ' ')) {
    jsonBuffer.remove(0, 1);
}
}

}

void HrScreensaver::playNextVideo() {
    if (!playlist.isEmpty()) {
        static int currentIndex = 0;

        if (currentIndex >= playlist.size()) {
            currentIndex = 0; // Сброс индекса в начало
        }

        QString path = playlist.at(currentIndex);
        if (QFile::exists(path)) {
            qDebug() << "Playing:" << path;
            player->setSource(QUrl::fromLocalFile(path));
            player->play();
            currentIndex++;
        } else {
            qDebug() << "File not found:" << path;
            playlist.removeAt(currentIndex);
            playNextVideo();
        }
    }
}

```

```

    } else {
        qDebug() << "Playlist is empty!";
        player->stop();
    }
}

void HrScreensaver::processJsonDocument(const QJsonDocument &doc) {
    if (!doc.isObject()) {
        qDebug() << "Expected JSON object in document";
        return;
    }
    QJsonObject jsonObj = doc.object();

    if (!jsonObj.contains("command")) {
        qDebug() << "JSON object missing 'command' field";
        return;
    }

    QString command = jsonObj["command"].toString();
    qDebug() << "Processing command:" << command;

    if (command == "content_list") {
        if (!jsonObj.contains("videos") || !jsonObj["videos"].isArray()) {
            qDebug() << "Invalid content_list command: missing or invalid
'videos' array";
            return;
        }
        playlist.clear();

        QJsonArray videosArray = jsonObj["videos"].toArray();
        for (const QJsonValue &videoValue : videosArray) {
            if (videoValue.isString()) {
                QString path = videoValue.toString();

                if (QFile::exists(path)) {
                    if (!playlist.contains(path)) {
                        playlist.append(path);
                        qDebug() << "Added to playlist:" << path;
                    } else {
                        qDebug() << "Duplicate video skipped:" << path;
                    }
                } else {

```

```

        qDebug() << "File not found:" << path;
    }
}

qDebug() << "Playlist updated. Total videos:" << playlist.size();

if (!playlist.isEmpty()) {
    isPlayingScreensaver = false;
    emptyPlayTimer->stop();
    playNextVideo();
} else {
    qDebug() << "Received empty content listm play Screen";
    playScreensaver();
}
}

else if (command == "add") {
    if (!jsonObj.contains("path")) {
        qDebug() << "'add' command missing 'path' field";
        return;
    }

    QString path = jsonObj["path"].toString();

    if (QFile::exists(path)) {
        if (!playlist.contains(path)) {
            playlist.append(path);
            qDebug() << "Added to playlist:" << path;
            if (player->mediaStatus() == QMediaPlayer::NoMedia) {
                playNextVideo();
            }
        } else {
            qDebug() << "Duplicate video skipped:" << path;
        }
    } else {
        qDebug() << "File not found:" << path;
    }
}

else if (command == "content_end") {
    qDebug() << "Content transfer completed. Total videos:" <<
playlist.size();

```

```

        if (!playlist.isEmpty() && player->mediaStatus() ==
QMediaPlayer::NoMedia) {
            playNextVideo();
        }
    }
    else if (command == "next") {
        qDebug() << "Received 'next' command";
        playNextVideo();
    }
    else if (command == "clear") {
        qDebug() << "Clearing playlist";
        playlist.clear();
        player->stop();
    }
    else {
        qDebug() << "Unknown command:" << command;
        qDebug() << "Full JSON:" << doc.toJson(QJsonDocument::Indented);
    }
}

HrScreensaver::~HrScreensaver() {
    qDebug() << "Cleaning up screensaver resources";

    if (player) {
        player->stop();
        player->setSource(QUrl());
        delete player;
        player = nullptr;
    }

    if (emptyPlayTimer) {
        emptyPlayTimer->stop();
        delete emptyPlayTimer;
        emptyPlayTimer = nullptr;
    }

    if (socket) {
        if (socket->state() != QLocalSocket::UnconnectedState) {
            socket->abort();
            socket->disconnectFromServer();
            if (socket->state() != QLocalSocket::UnconnectedState) {
                socket->waitForDisconnected(500);
            }
        }
    }
}

```

```

        }
        delete socket;
        socket = nullptr;
    }

```

```

    playlist.clear();

```

```

}

```

mainwindow.h

```

#ifndef MAINWINDOW_H
#define MAINWINDOW_H

```

```

#include <QMainWindow>
#include <QQtContext>
#include <QQuickWidget>
#include "hrscreensaver.h"
#include "userdatamodel.h"
#include <QLocalSocket>
#include <QSystemTrayIcon>
#include <QCloseEvent>

```

```

QT_BEGIN_NAMESPACE
namespace Ui {
class MainWindow;
}
QT_END_NAMESPACE

```

```

class MainWindow : public QMainWindow
{

```

```

    Q_OBJECT

```

```

public:

```

```

    MainWindow(QWidget *parent = nullptr);
    ~MainWindow();
    UserDataModel* userModel() const { return m_userModel; }

```

```

private:

```

```

    Ui::MainWindow *ui;
    HrScreensaver *screensaver = nullptr;
    QQuickWidget *qmlWidget;
    UserDataModel* m_userModel;
    QLocalSocket *serviceSocket;
    QSystemTrayIcon *trayIcon;
    void sendCommandToService(const QJsonObject& cmd);

```



```

        void closeEvent(QCloseEvent *event);

private slots:
    void onServiceConnected();
    void onServiceDisconnected();
    void onServiceReadyRead();
    void onServiceError(QLocalSocket::LocalSocketError error);
public slots:
    void startScreensaver();
    void saveUserData(const QString& firstName, const QString& lastName,
const QString& title, const QString& code);
    void fetchUserData();
};
#endif // MAINWINDOW_H

```

mainwindow.cpp

```

#include "mainwindow.h"
#include "ui_mainwindow.h"
#include <QPushButton>
#include <QJsonDocument>
#include <QJsonObject>
#include <QQuickItem>

MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
    , ui(new Ui::MainWindow)
    , m_userModel(new UserDataModel(this))
{
    ui->setupUi(this);

    setWindowTitle("HRScreenSaver");

    trayIcon = new QSystemTrayIcon(this);
    trayIcon->setIcon(QIcon(":/qml/qml/icon.png"));
    trayIcon->setToolTip("HRScreen");

    QMenu *menu = new QMenu(this);
    menu->addAction("Show", this, &MainWindow::show);
    menu->addAction("Hide", this, &MainWindow::hide);
    menu->addAction("Start Screensaver", this,
&MainWindow::startScreensaver);

```

```

menu->addSeparator();
menu->addAction("Exit", qApp, &QApplication::quit);

trayIcon->setContextMenu(menu);

QQuickWidget *qmlWidget = new QQuickWidget(this);
qmlWidget->setResizeMode(QQuickWidget::SizeRootObjectToView);
qmlWidget->rootContext()->setContextProperty("mainWindow", this);
qmlWidget->rootContext()->setContextProperty("userModel", m_userModel);
qmlWidget->setSource(QUrl("qrc:/qml/main.qml"));
setCentralWidget(qmlWidget);
resize(800, 600);
serviceSocket = new QLocalSocket(this);
connect(serviceSocket, &QLocalSocket::connected,
        this, &MainWindow::onServiceConnected);
connect(serviceSocket, &QLocalSocket::disconnected,
        this, &MainWindow::onServiceDisconnected);
connect(serviceSocket, &QLocalSocket::readyRead,
        this, &MainWindow::onServiceReadyRead);
connect(serviceSocket, &QLocalSocket::errorOccurred,
        this, &MainWindow::onServiceError);

connect(trayIcon, &QSystemTrayIcon::activated, this, [this](auto reason)
{
    if(reason == QSystemTrayIcon::DoubleClick) this->show();
});
trayIcon->show();
serviceSocket->connectToServer("hrscreensaver");
}

MainWindow::~MainWindow()
{
    if (serviceSocket && serviceSocket->state() ==
QLocalSocket::ConnectedState) {
        serviceSocket->disconnectFromServer();
    }

    if (centralWidget()) {
        QWidget* central = takeCentralWidget();
        delete central;
    }
}

```

```

        if (screensaver) {
            delete screensaver;
            screensaver = nullptr;
        }
        delete ui;
    }

void MainWindow::startScreensaver()
{
    serviceSocket->disconnectFromServer();
    qDebug() << "Starting screensaver...";
    if (!screensaver) {
        screensaver = new HrScreensaver();
    }
    screensaver->showFullScreen();
}

void MainWindow::onServiceConnected() {
    qDebug() << "Connected to service!";
    fetchUserData();
}

void MainWindow::onServiceDisconnected() {
    qDebug() << "Disconnected from service";
}

void MainWindow::onServiceError(QLocalSocket::LocalSocketError error) {
    qWarning() << "Service connection error:" << error;
}

void MainWindow::onServiceReadyRead() {
    QByteArray data = serviceSocket->readAll();
    QJsonDocument doc = QJsonDocument::fromJson(data);
    if (!doc.isObject()) return;

    QJsonObject obj = doc.object();
    QString command = obj["command"].toString();

    if (command == "user_data") {
        QString firstName = obj["first_name"].toString();
        QString lastName = obj["last_name"].toString();

        QQuickWidget* qmlWidget =
qobject_cast<QQuickWidget*>(centralWidget());
        if (qmlWidget) {

```

```

        QObject* root = static_cast<QObject*>(qmlWidget->rootObject());
        if (root) {
            QMetaObject::invokeMethod(root, "setUserData",
                                      Q_ARG(QVariant, firstName),
                                      Q_ARG(QVariant, lastName));
        }
    }
}

void MainWindow::sendCommandToService(const QJsonObject& cmd) {
    if (serviceSocket && serviceSocket->state() ==
        QLocalSocket::ConnectedState) {
        QJsonDocument doc(cmd);
        qDebug() << "sendCommandToService " << cmd;
        serviceSocket->write(doc.toJson());
        serviceSocket->flush();
    }
}

void MainWindow::fetchUserData() {
    QJsonObject cmd;
    cmd["command"] = "get_user_data";
    qDebug() << "fetchUserData " << cmd;
    sendCommandToService(cmd);
}

void MainWindow::saveUserData(const QString& firstName,
                              const QString& lastName,
                              const QString& title,
                              const QString& code)
{
    QJsonObject cmd;
    cmd["command"] = "set_user_data";
    cmd["first_name"] = firstName;
    cmd["last_name"] = lastName;
    cmd["title"] = title;
    cmd["code"] = code;
    sendCommandToService(cmd);
}

void MainWindow::closeEvent(QCloseEvent *event)
{
    this->hide();
    event->ignore();
}

```

```
}
```

main.cpp

```
#include "mainwindow.h"
```

```
#include <QApplication>
```

```
#include <QLocale>
```

```
#include <QTranslator>
```

```
#include <QCommandLineParser>
```

```
#include "hrscreensaver.h"
```

```
#include <QQuickStyle>
```

```
#include <QStringList>
```

```
#include <QString>
```

```
#include <QDebug>
```

```
#include <QMutexLocker>
```

```
#include <QStandardPaths>
```

```
#include <QFile>
```

```
#include <QDir>
```

```
#ifdef Q_OS_WIN
```

```
#include <windows.h>
```

```
#endif
```

```
static QMutex logMutex;
```

```
void messageHandler(QtMsgType type, const QMessageLogContext &context, const  
QString &msg)
```

```
{
```

```
    QMutexLocker locker(&logMutex);
```

```
    QString logFilePath =
```

```
QStandardPaths::writableLocation(QStandardPaths::AppDataLocation) +  
"/app_log.txt";
```

```
    QFile logFile(logFilePath);
```

```
    if (!logFile.open(QIODevice::WriteOnly | QIODevice::Append |  
QIODevice::Text)) {
```

```
        return;
```

```
    }
```

```
    QTextStream out(&logFile);
```

```
    out.setEncoding(QStringConverter::Utf8);
```

```
    QString timestamp = QDateTime::currentDateTime().toString("yyyy-MM-dd  
hh:mm:ss.zzz");
```

```
    QString typeStr;
```

```
    switch (type) {
```

```

        case QtDebugMsg:    typeStr = "DEBUG"; break;
        case QtInfoMsg:     typeStr = "INFO"; break;
        case QtWarningMsg:  typeStr = "WARNING"; break;
        case QtCriticalMsg: typeStr = "CRITICAL"; break;
        case QtFatalMsg:    typeStr = "FATAL"; break;
    }

    QString logLine = QString("[%1] [%2] %3 - %4 (%5:%6, %7)\n")
        .arg(timestamp)
        .arg(typeStr)
        .arg(context.category)
        .arg(msg)
        .arg(context.file)
        .arg(context.line)
        .arg(context.function);

    out << logLine;
    out.flush();
}

int main(int argc, char *argv[])
{
    QQuickStyle::setStyle("Material");
    qInstallMessageHandler(messageHandler);
    QApplication a(argc, argv);
    QCommandLineParser parser;
    parser.setApplicationDescription("HR ScreenSaver Application");
    parser.addHelpOption();
    QCommandLineOption screensaverOption("s", "Run in screensaver mode");
    parser.addOption(screensaverOption);
    parser.process(a);
    bool isScr = false;
    QString logDir =
QStandardPaths::writableLocation(QStandardPaths::AppDataLocation);
    QDir().mkpath(logDir);
    qDebug() << "LOG: " << logDir;
    QTranslator translator;
    const QStringList uiLanguages = QLocale::system().uiLanguages();
    for (const QString &locale : uiLanguages) {
        const QString baseName = "HRScreenSaver_" + QLocale(locale).name();
        if (translator.load(":/i18n/" + baseName)) {
            a.installTranslator(&translator);
            break;
        }
    }
}

```

```

    }
    MainWindow w;

    if (parser.isSet(screensaverOption)) {
        qDebug() << "Starting screensaver in full-screen mode";
        w.startScreensaver();
        return a.exec();
    }
    QStringList args = QApplication::arguments();
    isScr = args.at(0).endsWith(".scr", Qt::CaseInsensitive);
    qDebug() << "arg == " << args;
    if (args.size() > 1) {
        qDebug() << "args.size() " << args.size();
        QString arg = args.at(1);
        qDebug() << "arg == " << arg;
        if (arg == "/c" || arg == "/C") {
            w.show();
        } else if (arg == "/s" || arg == "/S") {
            qDebug() << "arg is /s == " << arg;
            w.startScreensaver();
            return a.exec();
        }
    }
    w.show();
    return a.exec();
}

```

Исходный код клиентской части по работе с базой данных и сетью.

databasemanager.h

```

#ifndef DATABASEMANAGER_H
#define DATABASEMANAGER_H
#include <QObject>
#include <QSqlDatabase>
#include <QJsonObject>
#include <QJsonArray>
#include "networkmanager.h"
#ifndef COMMONMODELS_LIBRARY
#define COMMONMODELS_EXPORT Q_DECL_IMPORT
#else
#define COMMONMODELS_EXPORT Q_DECL_EXPORT
#endif

```

```

class COMMONMODELS_EXPORT DatabaseManager : public QObject
{
    Q_OBJECT
public:
    explicit DatabaseManager(QObject *parent = nullptr);
    ~DatabaseManager();
    bool initDatabase();
    void saveDataFromJson(const QByteArray &jsonData);
    void cleanupOldData();
    void downloadVideos();
    QString getVideoStoragePath() const;
    QStringList getCurrentVideos();
    void saveUserData(const QString& firstName, const QString& lastName,
const QString& title, const QString& code);
    QString getUserFirstName();
    QString getUserLastName();
    QString getDepartment();
    QString getDepartmentCode();
signals:
    void videoDownloaded(const QString& localPath);
    void allDownloadsFinished();

private:
    QSqlDatabase db;
    NetworkManager* networkManager;
    QString videoStoragePath;
    QString getDatabasePath() const;
    void createDebugDirectory() const;
    void saveDownloadedVideo(int contentId, const QString& localPath);
    void savePlaylist(const QJsonObject &playlist);
    void saveContent(int playlistId, const QJsonObject &content);
    void saveSchedule(int contentId, const QJsonObject &schedule);
    void saveWeekdays(int scheduleId, const QJsonArray &weekdays);
    QJsonObject loadUserData();
private slots:
    void handleDownloadFinished(int contentId, const QString& localPath);
};

```

```
#endif // DATABASEMANAGER_H
```

databasemanager.cpp

```
#include "databasemanager.h"
```



```

#include <QDebug>
#include <QDir>
#include <QStandardPaths>
#include <QSqlQuery>
#include <QSqlError>
#include <QJsonDocument>
#include <QJsonParseError>
#include <QJsonArray>
#include <QDate>
#include <QTime>
#include <QTimer>

DatabaseManager::DatabaseManager(QObject *parent) : QObject(parent) {
    networkManager = new NetworkManager(this);
    videoStoragePath = getVideoStoragePath();
    initDatabase();
    connect(networkManager, &NetworkManager::downloadFinished,
            this, &DatabaseManager::handleDownloadFinished);
    connect(networkManager, &NetworkManager::allDownloadsFinished,
            this, &DatabaseManager::allDownloadsFinished);
}

DatabaseManager::~DatabaseManager() {
    if (db.isOpen()) {
        db.close();
    }
}

bool DatabaseManager::initDatabase() {
    QString dbPath =
QStandardPaths::writableLocation(QStandardPaths::AppDataLocation);
    QDir().mkpath(dbPath);
    dbPath += "/hrscreen.db";
    qDebug() << "Database path:" << dbPath;

    db = QSqlDatabase::addDatabase("QSQLITE");
    db.setDatabaseName(dbPath);

    if (!db.open()) {
        qDebug() << "Database error:" << db.lastError().text();
        return false;
    }

    QSqlQuery query;
    bool success = true;

```

```

success &= query.exec("CREATE TABLE IF NOT EXISTS playlists ("
    "id INTEGER PRIMARY KEY, "
    "name TEXT, "
    "playOrder INTEGER) "
    ");

success &= query.exec("CREATE TABLE IF NOT EXISTS content ("
    "id INTEGER PRIMARY KEY, "
    "playlist_id INTEGER, "
    "title TEXT, "
    "path TEXT, "
    "localpath TEXT, "
    "playOrder INTEGER, "
    "startDay TEXT, "
    "endDay TEXT, "
    "FOREIGN KEY(playlist_id) REFERENCES
playlists(id))");

success &= query.exec("CREATE TABLE IF NOT EXISTS schedules ("
    "id INTEGER PRIMARY KEY, "
    "schedule_id INTEGER, "
    "content_id INTEGER, "
    "startTime TEXT, "
    "endTime TEXT, "
    "FOREIGN KEY(content_id) REFERENCES content(id))");

success &= query.exec("CREATE TABLE IF NOT EXISTS schedule_weekdays ("
    "schedule_id INTEGER, "
    "weekday TEXT, "
    "PRIMARY KEY(schedule_id, weekday), "
    "FOREIGN KEY(schedule_id) REFERENCES
schedules(id))");

success &= query.exec("CREATE TABLE IF NOT EXISTS userdata ("
    "id INTEGER PRIMARY KEY, "
    "firstName TEXT, "
    "lastName TEXT, "
    "department TEXT, "
    "departmentcode TEXT) ");

```

```

        query.exec("CREATE INDEX IF NOT EXISTS idx_content_dates ON
content(startDay, endDay)");

        query.exec("CREATE INDEX IF NOT EXISTS idx_schedules_times ON
schedules(startTime, endTime)");

        if (!success) {
            qDebug() << "Database initialization error:" <<
query.lastError().text();
        }
        else{
            qDebug() << "Database initialization!!";
        }
        return success;
    }

void DatabaseManager::saveDataFromJson(const QByteArray &jsonData) {
    QJsonParseError error;
    QJsonDocument doc = QJsonDocument::fromJson(jsonData, &error);
    if (error.error != QJsonParseError::NoError) {
        qDebug() << "JSON Parse Error:" << error.errorString();
        return;
    }
    if (!doc.isArray() || doc.array().isEmpty()) {
        qDebug() << "Expected non-empty JSON array in response";
        return;
    }
    QJsonObject mainObject = doc.array().first().toObject();
    QJsonArray playlists = mainObject["playLists"].toArray();
    qDebug() << "Number of playlists received:" << playlists.size();
    db.transaction();
    QSqlQuery clearQuery(db);
    if (!clearQuery.exec("DELETE FROM content")) {
        qDebug() << "Failed to clear table:" <<
clearQuery.lastError().text();
        db.rollback();
        return;
    }
    for (const QJsonValue &playlistValue : playlists) {
        if (playlistValue.isObject()) {
            savePlaylist(playlistValue.toObject());
        }
    }
}

```

```

    }

    if (!db.commit()) {
        qDebug() << "Transaction commit error:" << db.lastError().text();
        db.rollback();
    } else {
        qDebug() << "Data successfully saved to database";

        QTimer::singleShot(0, this, &DatabaseManager::downloadVideos);
    }
}

void DatabaseManager::savePlaylist(const QJsonObject &playlist) {
    QSqlQuery query;
    query.prepare("INSERT OR REPLACE INTO playlists (id, name, playOrder) "
                  "VALUES (:id, :name, :playOrder)");
    query.bindValue(":id", playlist["id"].toInt());
    query.bindValue(":name", playlist["name"].toString());
    query.bindValue(":playOrder", playlist["playOrder"].toInt());

    if (!query.exec()) {
        qDebug() << "Error saving playlist:" << query.lastError().text();
        return;
    }

    int playlistId = playlist["id"].toInt();
    QJsonArray content = playlist["content"].toArray();
    for (const QJsonValue &contentValue : content) {
        if (contentValue.isObject()) {
            saveContent(playlistId, contentValue.toObject());
        }
    }
}

void DatabaseManager::saveContent(int playlistId, const QJsonObject &content)
{
    QSqlQuery query;
    query.prepare("INSERT OR REPLACE INTO content (id, playlist_id, title,
path, playOrder, startDay, endDay) "
                  "VALUES (:id, :playlist_id, :title, :path, :playOrder,
:startDay, :endDay)");
    query.bindValue(":id", content["id"].toInt());
    query.bindValue(":playlist_id", playlistId);
    query.bindValue(":title", content["title"].toString());

```

```

        query.bindValue(":path", content["path"].toString());
        query.bindValue(":playOrder", content["playOrder"].toInt());
        query.bindValue(":startDay", content["startDay"].toString());
        query.bindValue(":endDay", content["endDay"].toString());
        if (!query.exec()) {
            qDebug() << "Error saving content:" << query.lastError().text();
            return;
        }
        int contentId = content["id"].toInt();
        QJsonObject schedule = content["schedule"].toObject();
        saveSchedule(contentId, schedule);
    }
}

void DatabaseManager::saveSchedule(int contentId, const QJsonObject
&schedule) {
    QSqlQuery query;

    query.prepare("INSERT OR REPLACE INTO schedules (id, schedule_id,
content_id, startTime, endTime) "
                "VALUES (:id, :schedule_id, :content_id, :startTime,
:endTime)");
    query.bindValue(":id", contentId );
    query.bindValue(":schedule_id", schedule["id"].toInt() );
    query.bindValue(":content_id", contentId);
    query.bindValue(":startTime", schedule["startTime"].toString());
    query.bindValue(":endTime", schedule["endTime"].toString());

    if (!query.exec()) {
        qDebug() << "Error saving schedule:" << query.lastError().text();
        return;
    }

    int scheduleId = schedule["id"].toInt();
    QJsonArray weekdays = schedule["weekdays"].toArray();
    saveWeekdays(scheduleId, weekdays);
}

void DatabaseManager::saveWeekdays(int scheduleId, const QJsonArray
&weekdays) {
    QSqlQuery query;
    query.prepare("INSERT OR REPLACE INTO schedule_weekdays (schedule_id,
weekday) "
                "VALUES (:schedule_id, :weekday)");

    for (const QJsonValue &weekdayValue : weekdays) {

```

```

        query.bindValue(":schedule_id", scheduleId);
        query.bindValue(":weekday", weekdayValue.toString());
        if (!query.exec()) {
            qDebug() << "Error saving weekday:" << query.lastError().text();
        }
    }
}

void DatabaseManager::cleanupOldData() {
    QSqlQuery query;
    QString currentDate = QDate::currentDate().toString(Qt::ISODate);
    query.prepare("DELETE FROM content WHERE endDay < :currentDate");
    query.bindValue(":currentDate", currentDate);
    if (!query.exec()) {
        qDebug() << "Error cleaning up old content:" <<
query.lastError().text();
    }

    query.exec("DELETE FROM schedules WHERE content_id NOT IN (SELECT id FROM
content)");
    query.exec("DELETE FROM schedule_weekdays WHERE schedule_id NOT IN
(SELECT id FROM schedules)");
    query.exec("DELETE FROM playlists WHERE id NOT IN (SELECT playlist_id
FROM content)");
    query.exec("DELETE FROM channels WHERE id NOT IN (SELECT channel_id FROM
playlists)");
}

QString DatabaseManager::getVideoStoragePath() const {
    QString path =
QStandardPaths::writableLocation(QStandardPaths::AppDataLocation);
    return path + "/videos";
}

void DatabaseManager::downloadVideos() {
    QSqlQuery query;
    query.prepare("SELECT id, path FROM content WHERE path LIKE 'http%'");
    if (!query.exec()) {
        qDebug() << "Error querying videos to download:" <<
query.lastError().text();
        return;
    }

    QList<QPair<int, QString>> videosToDownload;

```

```

while (query.next()) {
    int contentId = query.value(0).toInt();
    QString url = query.value(1).toString();
    videosToDownload.append(qMakePair(contentId, url));
}

QDebug() << "Found" << videosToDownload.size() << "videos to download";

networkManager->downloadVideos(videosToDownload, videoStoragePath);
}

void DatabaseManager::handleDownloadFinished(int contentId, const QString&
localPath) {
    saveDownloadedVideo(contentId, localPath);

    emit videoDownloaded(localPath);
}

void DatabaseManager::saveDownloadedVideo(int contentId, const QString&
localPath) {
    QSqlQuery query;
    query.prepare("UPDATE content SET localpath = :localpath WHERE id =
:id");
    query.bindValue(":localpath", localPath);
    query.bindValue(":id", contentId);

    if (!query.exec()) {
        qDebug() << "Error updating video path:" << query.lastError().text();
    } else {
        qDebug() << "Updated localpath for content ID" << contentId << "to"
<< localPath;
    }
}

QStringList DatabaseManager::getCurrentVideos() {
    QStringList videos;
    QDate currentDate = QDate::currentDate();
    QTime currentTime = QTime::currentTime();
    int currentWeekday = currentDate.dayOfWeek() % 7;

    QSqlQuery query;
    query.prepare(
        "SELECT c.localpath FROM content c "
        "JOIN schedules s ON c.id = s.content_id "
        "JOIN schedule_weekdays w ON s.schedule_id = w.schedule_id "

```

```

        "WHERE :currentDate BETWEEN c.startDay AND c.endDay "
        "AND :currentTime BETWEEN s.startTime AND s.endTime "
        "AND w.weekday = :weekday "
        "ORDER BY c.playOrder"
    );
    query.bindValue(":currentDate", currentDate.toString(Qt::ISODate));
    query.bindValue(":currentTime", currentTime.toString("HH:mm:ss"));
    query.bindValue(":weekday", QString::number(currentWeekday));
    QString debugQuery = QString(
        "SELECT c.localpath FROM content c "
        "JOIN schedules s ON c.id = s.content_id "
        "JOIN schedule_weekdays w ON s.schedule_id =
w.schedule_id "

        "WHERE %1 BETWEEN c.startDay AND c.endDay "
        "AND %2 BETWEEN s.startTime AND s.endTime "
        "AND w.weekday = %3 "
        "ORDER BY c.playOrder"
    ).arg(currentDate.toString(Qt::ISODate))
        .arg(currentTime.toString("HH:mm:ss"))
        .arg(currentWeekday);

    qDebug() << "Executing query:" << debugQuery;
    if (!query.exec()) {
        qDebug() << "Error querying current videos:" <<
query.lastError().text();
        return videos;
    }

    while (query.next()) {
        videos.append(query.value(0).toString());
    }
    qDebug() << "query VIDEO" << videos ;
    return videos;
}

void DatabaseManager::saveUserData(const QString& firstName,
                                   const QString& lastName,
                                   const QString& title,
                                   const QString& code)
{
    QSqlQuery query;

```



```

        query.prepare("INSERT OR REPLACE INTO userdata (id, firstName, lastName,
department, departmentcode ) "
                        "VALUES (1, :firstName, :lastName, :department,
:departmentcode )");
        query.bindValue(":firstName", firstName);
        query.bindValue(":lastName", lastName);
        query.bindValue(":department", title);
        query.bindValue(":departmentcode", code);

        if (!query.exec()) {
            qDebug() << "Error querying user data:" << query.lastError().text();
        }
    }

QString DatabaseManager::getUserFirstName()
{
    QString value;
    QSqlQuery query;
    query.prepare("SELECT firstName FROM userdata");

    if (!query.exec()) {
        qDebug() << "Error getting user data:" << query.lastError().text();
        return value;
    }

    if (query.next()) {
        value = query.value(0).toString();
    }

    return value;
}

QString DatabaseManager::getDepartment()
{
    QString value;
    QSqlQuery query;
    query.prepare("SELECT department FROM userdata");

    if (!query.exec()) {
        qDebug() << "Error getting user data:" << query.lastError().text();
        return value;
    }

    if (query.next()) {

```

```

        value = query.value(0).toString();
    }
    return value;
}

QString DatabaseManager::getDepartmentCode()
{
    QString value;
    QSqlQuery query;
    query.prepare("SELECT departmentcode FROM userdata");

    if (!query.exec()) {
        qDebug() << "Error getting user data:" << query.lastError().text();
        return value;
    }
    if (query.next()) {
        value = query.value(0).toString();
    }
    return value;
}

QString DatabaseManager::getUserLastName()
{
    QString value;
    QSqlQuery query;
    query.prepare("SELECT lastName FROM userdata");

    if (!query.exec()) {
        qDebug() << "Error getting user data:" << query.lastError().text();
        return value;
    }
    if (query.next()) {
        value = query.value(0).toString();
    }
    return value;
}

QJsonObject DatabaseManager::loadUserData() {
    QJsonObject result;
    QSqlQuery query("SELECT first_name, last_name FROM user_data WHERE id =
1");
    if (query.next()) {
        result["first_name"] = query.value(0).toString();
        result["last_name"] = query.value(1).toString();
    }
}

```

```

        return result;
    }

```

networkmanager.h

```

#ifndef NETWORKMANAGER_H
#define NETWORKMANAGER_H
#include <QObject>
#include <QNetworkAccessManager>
#include <QNetworkReply>
#include <QFile>
#include <QQueue>
#include <QPair>
#ifndef COMMONMODELS_LIBRARY
#define COMMONMODELS_EXPORT Q_DECL_IMPORT
#else
#define COMMONMODELS_EXPORT Q_DECL_EXPORT
#endif

class COMMONMODELS_EXPORT NetworkManager : public QObject
{
    Q_OBJECT
public:
    explicit NetworkManager(QObject *parent = nullptr);
    ~NetworkManager();

    void downloadVideos(const QList<QPair<int, QString>>& videos, const
QString& storagePath);
    void cancelDownloads();

signals:
    void downloadProgress(int contentId, qint64 bytesReceived, qint64
bytesTotal);
    void downloadFinished(int contentId, const QString& localPath);
    void downloadFailed(int contentId, const QString& error);
    void allDownloadsFinished();

private:
    QNetworkAccessManager* manager;
    QQueue<QPair<int, QString>> downloadQueue;
    QString videoStoragePath;
    bool isDownloading;

    void startNextDownload();

```

```

private slots:
    void onDownloadProgress(qint64 bytesReceived, qint64 bytesTotal);
    void onDownloadFinished();
};

#endif // NETWORKMANAGER_H

networkmanager.cpp

#include "networkmanager.h"
#include <QDebug>
#include <QFileInfo>
#include <QDir>
#include <QNetworkRequest>
#include <QNetworkReply>
NetworkManager::NetworkManager(QObject *parent)
    : QObject(parent), manager(new QNetworkAccessManager(this)),
    isDownloading(false)
{
}

NetworkManager::~NetworkManager() {
    cancelDownloads();
}

void NetworkManager::downloadVideos(const QList<QPair<int, QString>>& videos,
const QString& storagePath) {
    if (videos.isEmpty()) {
        emit allDownloadsFinished();
        return;
    }
    cancelDownloads();
    videoStoragePath = storagePath;
    QDir dir(videoStoragePath);
    if (!dir.exists()) {
        dir.mkpath(".");
    }
    downloadQueue.clear();
    for (const auto& video : videos) {
        downloadQueue.enqueue(video);
    }
    qDebug() << "Starting download of" << downloadQueue.size() << "videos";
    isDownloading = true;
    startNextDownload();
}

```

```

void NetworkManager::cancelDownloads() {
    if (isDownloading) {
        for (QNetworkReply* reply : manager->findChildren<QNetworkReply*>())
        {
            reply->abort();
            reply->deleteLater();
        }

        downloadQueue.clear();
        isDownloading = false;
        qDebug() << "Downloads canceled";
    }
}

void NetworkManager::startNextDownload() {
    if (downloadQueue.isEmpty()) {
        isDownloading = false;
        emit allDownloadsFinished();
        return;
    }
    auto item = downloadQueue.dequeue();
    int contentId = item.first;
    QString url = item.second;

    QNetworkRequest request(url);
    QNetworkReply* reply = manager->get(request);

    reply->setProperty("contentId", contentId);
    reply->setProperty("url", url);

    connect(reply, &QNetworkReply::downloadProgress,
            this, &NetworkManager::onDownloadProgress);
    connect(reply, &QNetworkReply::finished,
            this, &NetworkManager::onDownloadFinished);
}

void NetworkManager::onDownloadProgress(qint64 bytesReceived, qint64
bytesTotal) {
    QNetworkReply* reply = qobject_cast<QNetworkReply*>(sender());
    if (!reply) return;

    int contentId = reply->property("contentId").toInt();
    emit downloadProgress(contentId, bytesReceived, bytesTotal);
}

```

```

void NetworkManager::onDownloadFinished() {
    QNetworkReply* reply = qobject_cast<QNetworkReply*>(sender());
    if (!reply) return;
    int contentId = reply->property("contentId").toInt();
    QString url = reply->property("url").toString();
    if (reply->error() != QNetworkReply::NoError) {
        QString error = reply->errorString();
        qDebug() << "Download failed for" << url << ":" << error;
        emit downloadFailed(contentId, error);

        startNextDownload();
        reply->deleteLater();
        return;
    }
    QString fileName = QFileInfo(url).fileName();
    QString localPath = videoStoragePath + "/" + fileName;
    QFile file(localPath);
    if (file.open(QIODevice::WriteOnly)) {
        file.write(reply->readAll());
        file.close();
        qDebug() << "Downloaded video:" << localPath;
        emit downloadFinished(contentId, localPath);
    } else {
        QString error = "Failed to save file: " + localPath;
        qDebug() << error;
        emit downloadFailed(contentId, error);
    }
    startNextDownload();
    reply->deleteLater();
}

```

userdatamodel.h

```

#ifndef USERDATAMODEL_H
#define USERDATAMODEL_H
#include <QObject>
#include <QString>
#ifndef COMMONMODELS_LIBRARY
#define COMMONMODELS_EXPORT Q_DECL_IMPORT
#else
#define COMMONMODELS_EXPORT Q_DECL_EXPORT
#endif
class COMMONMODELS_EXPORT UserDataModel : public QObject
{

```

```

    Q_OBJECT
    Q_PROPERTY(QString firstName READ firstName WRITE setFirstName NOTIFY
firstNameChanged)
    Q_PROPERTY(QString lastName READ lastName WRITE setLastName NOTIFY
lastNameChanged)
    Q_PROPERTY(QString department READ department WRITE setDepartment NOTIFY
departmentChanged)
    Q_PROPERTY(QString departmentCode READ departmentCode WRITE
setDepartmentCode NOTIFY departmentCodeChanged)

public:
    explicit UserDataModel(QObject *parent = nullptr);
    QString firstName() const;
    QString lastName() const;
    QString department() const;
    QString departmentCode() const;
    QString getIpAddresses();
    QString getCurrentUserName();
    QString getDeviceId();
    void setFirstName(const QString &firstName);
    void setLastName(const QString &lastName);
    void setDepartment(const QString &department);
    void setDepartmentCode(const QString &departmentCode);
    QJsonObject toJson() const;
    void fromJson(const QJsonObject &json);

signals:
    void firstNameChanged(const QString &firstName);
    void lastNameChanged(const QString &lastName);
    void departmentChanged(const QString &department);
    void departmentCodeChanged(const QString &departmentCode);
    void dataChanged();

private:
    QString m_firstName;
    QString m_lastName;
    QString m_department;
    QString m_departmentCode;
};

#endif // USERDATAMODEL_H

```

userdatamodel.cpp

```

#include "userdatamodel.h"
#include <QJsonObject>

```

```

#include <QNetworkInterface>
#include <QCryptographicHash>
#include <QSettings>
#include <QProcess>
#include <QUuid>
#include <QStringList>

#ifdef Q_OS_WIN
#include <windows.h>
#include <lmcons.h>
#include <wincrypt.h>
#endif

UserDataModel::UserDataModel(QObject *parent) : QObject(parent) {}

QString UserDataModel::firstName() const {
    return m_firstName;
}

QString UserDataModel::lastName() const {
    return m_lastName;
}

QString UserDataModel::department() const {
    return m_department;
}

QString UserDataModel::departmentCode() const {
    return m_departmentCode;
}

void UserDataModel::setFirstName(const QString &firstName) {
    if (m_firstName != firstName) {
        m_firstName = firstName;
        emit firstNameChanged(m_firstName);
        emit dataChanged();
    }
}

void UserDataModel::setLastName(const QString &lastName) {
    if (m_lastName != lastName) {
        m_lastName = lastName;
        emit lastNameChanged(m_lastName);
        emit dataChanged();
    }
}

void UserDataModel::setDepartment(const QString &department) {
    if (m_department != department) {
        m_department = department;
    }
}

```



```

        emit departmentChanged(m_department);
        emit dataChanged();
    }
}

void UserDataModel::setDepartmentCode(const QString &departmentCode) {
    if (m_departmentCode != departmentCode) {
        m_departmentCode = departmentCode;
        emit departmentCodeChanged(m_departmentCode);
        emit dataChanged();
    }
}

QJsonObject UserDataModel::toJson() const {
    QJsonObject obj;
    obj["first_name"] = m_firstName;
    obj["last_name"] = m_lastName;
    return obj;
}

void UserDataModel::fromJson(const QJsonObject &json) {
    setFirstName(json.value("first_name").toString());
    setLastName(json.value("last_name").toString());
    setDepartment(json.value("department").toString());
    setDepartmentCode(json.value("departmentCode").toString());
}

QString UserDataModel::getIpAddresses() {
    QStringList ipList;
    const QList<QNetworkInterface> interfaces =
QNetworkInterface::allInterfaces();

    for (int i = 0; i < interfaces.size(); ++i) {
        const QNetworkInterface &inter = interfaces.at(i);
        bool isUp = inter.flags() & QNetworkInterface::IsUp;
        bool isLoopBack = inter.flags() & QNetworkInterface::IsLoopBack;

        if (!isUp || isLoopBack) {
            continue;
        }
        const QList<QNetworkAddressEntry> entries = inter.addressEntries();
        for (int j = 0; j < entries.size(); ++j) {
            const QNetworkAddressEntry &entry = entries.at(j);
            if (entry.ip().protocol() == QAbstractSocket::IPv4Protocol) {
                ipList << entry.ip().toString();
            }
        }
    }
}

```

```

        }
    }
}

if (!ipList.isEmpty()) {
    return ipList[0];
}

return "unknown";
}

QString UserDataModel::getCurrentUserName() {
#ifdef Q_OS_WIN
    wchar_t username[UNLEN + 1];
    DWORD username_len = UNLEN + 1;
    if (GetUserNameW(username, &username_len)) {
        return QString::fromWCharArray(username);
    }
#endif
    return qgetenv("USERNAME");
}

QString UserDataModel::getDeviceId() {
    QSettings settings;
    QString uniqueData;
    QString deviceId = settings.value("device_id").toString();
    if (!deviceId.isEmpty()) {
        return deviceId;
    }

#ifdef Q_OS_WIN
    QProcess process;
    process.start("wmic", QStringList() << "bios" << "get" <<
"serialnumber");
    if (process.waitForFinished()) {
        uniqueData +=
QString::fromLocal8Bit(process.readAllStandardOutput());
    }
#endif

#ifdef Q_OS_WIN
    HKEY hKey;
    if (RegOpenKeyExW(HKEY_LOCAL_MACHINE,
L"SOFTWARE\\Microsoft\\Cryptography",

```

```

        0, KEY_READ, &hKey) == ERROR_SUCCESS) {
    wchar_t value[255];
    DWORD size = sizeof(value);
    if (RegQueryValueExW(hKey, L"MachineGuid", nullptr, nullptr,
        reinterpret_cast<LPBYTE>(value), &size) ==
ERROR_SUCCESS) {
        uniqueData += QString::fromWCharArray(value);
    }
    RegCloseKey(hKey);
}
#endif

if (uniqueData.isEmpty()) {
    deviceId = QUuid::createUuid().toString();
} else {
    QCryptographicHash hash(QCryptographicHash::Sha256);
    hash.addData(uniqueData.toUtf8());
    deviceId = hash.result().toHex();
}

settings.setValue("device_id", deviceId);
return deviceId;
}

```

Исходный код клиентской части сервиса загрузки.

service.h

```

#ifndef SERVICE_H
#define SERVICE_H

#include <QObject>
#include <QLocalServer>
#include <QLocalSocket>
#include <QNetworkAccessManager>
#include <QTimer>
#include "databasemanager.h"
#include "userdatamodel.h"
class Service : public QObject {
    Q_OBJECT
public:
    explicit Service(const QString& serverUrl = "", QObject *parent =
nullptr);    ~Service();
    UserDataModel* userModel() const { return m_userModel; }

```

```

        void startCheckingUpdates();
        void sendUserDataToClients(const QJsonObject& data);
private:
    QLocalServer *server;
    QList<QLocalSocket*> clients;
    QNetworkAccessManager downloadManager;
    DatabaseManager* dbManager;
    QNetworkAccessManager* networkManager;
    QTimer* updateTimer;
    QString apiServerUrl;
    UserDataModel* m_userModel;
    void onNewConnection();
private slots:
    void fetchUpdatesFromServer();
    void onServerResponseReceived(QNetworkReply* reply);
    void onUpdateTimerTimeout();
    void sendContentToClient(QLocalSocket* client);
    void saveDataUser();
};
#endif // SERVICE_H

```

service.cpp

```

#include "service.h"
#include <QDebug>
#include <QFile>
#include <QJsonDocument>
#include <QJsonObject>
#include <QNetworkRequest>
#include <QUrl>

Service::Service(const QString& serverUrl, QObject *parent) :
    QObject(parent),
    dbManager(new DatabaseManager(this)),
    apiServerUrl(serverUrl), m_userModel(new UserDataModel(this)) {
    server = new QLocalServer(this);
    networkManager = new QNetworkAccessManager(this);
    updateTimer = new QTimer(this);
    updateTimer->setInterval(5 * 60 * 1000);
    QLocalServer::removeServer("hrscreen_screensaver");
    if (!server->listen("hrscreen_screensaver")) {
        qDebug() << "Server error:" << server->errorString();
    }
}

```

```

        connect(server, &QLocalServer::newConnection, this,
&Service::onNewConnection);
        connect(updateTimer, &QTimer::timeout, this,
&Service::onUpdateTimerTimeout);
        connect(m_userModel, &UserDataModel::dataChanged, this,
&Service::saveDataUser);
    }
Service::~Service() {
    updateTimer->stop();
}

void Service::onNewConnection() {
    QLocalSocket *client = server->nextPendingConnection();
    clients.append(client);

    connect(client, &QLocalSocket::readyRead, [this, client]() {
        QByteArray data = client->readAll();
        QJsonDocument doc = QJsonDocument::fromJson(data);
        if (doc.isObject()) {
            QJsonObject cmd = doc.object();
            QString command = cmd["command"].toString();
            if (command == "get_content") {
                sendContentToClient(client);
            }
            else if (command == "get_user_data") {
                QJsonObject response;
                response["command"] = "user_data";
                response["first_name"] = dbManager->getUserFirstName();
                response["last_name"] = dbManager->getUserLastName();
                response["title"] = dbManager->getDepartment();
                response["code"] = dbManager->getDepartmentCode();
                QJsonDocument doc(response);
                qDebug() << "Response Command " << doc;
                client->write(doc.toJson());
                client->flush();
            }
            else if (command == "set_user_data") {
                // Обновление данных от клиента
                m_userModel->setFirstName(cmd["first_name"].toString());
                m_userModel->setLastName(cmd["last_name"].toString());
                m_userModel->setDepartment(cmd["title"].toString());
                m_userModel->setDepartmentCode(cmd["code"].toString());
                qDebug() << "User data updated:"

```

```

        << m_userModel->firstName() << m_userModel->
>lastName() << m_userModel->department() << m_userModel->departmentCode();

        dbManager->saveUserData(m_userModel->firstName()
                                , m_userModel->lastName()
                                , m_userModel->department()
                                , m_userModel->departmentCode());
        qDebug() << "User data save DB:"
        << m_userModel->firstName() << m_userModel->
>lastName();

        startCheckingUpdates();
    }
}

});

connect(client, &QLocalSocket::disconnected, [this, client]() {
    clients.removeAll(client);
    client->deleteLater();
});
}

void Service::startCheckingUpdates() {
    if (apiServerUrl.isEmpty()) {
        qWarning() << "API server URL not set!";
        return;
    }
    fetchUpdatesFromServer();
    updateTimer->start();
}

void Service::fetchUpdatesFromServer() {
    QString code = dbManager->getDepartmentCode();
    QString ip = m_userModel->getIpAddresses();
    QString username = m_userModel->getCurrentUserName().toUtf8();
    QString deviceId = m_userModel->getDeviceId();
    QString lastname = dbManager->getUserLastName().toUtf8();
    QString firstname = dbManager->getUserFirstName().toUtf8();
    if (code != ""){
        QString back = apiServerUrl;
        apiServerUrl = apiServerUrl+code;
        apiServerUrl = apiServerUrl + "&ip=" + ip
            + "&login=" + username
            + "&deviceId=" + deviceId
            + "&lastname=" + lastname

```

```

        + "&firstname=" + firstname;
    QUrl url(apiServerUrl);
    qDebug() << "URL: " << apiServerUrl;
    apiServerUrl = back;
    if (!url.isValid()) {
        qDebug() << "Invalid URL:" << apiServerUrl;
        return;
    }
    QNetworkRequest request;
    request.setUrl(url);
    request.setHeader(QNetworkRequest::ContentTypeHeader,
"application/json");
    QString token = "230875e6-d857-4853-a6b0-0aac83fa9142";
    request.setRawHeader("Authorization", ("Token " + token).toUtf8());
    QNetworkReply* reply = networkManager->get(request);
    connect(reply, &QNetworkReply::finished,
            this, [this, reply]() { this-
>onServerResponseReceived(reply); });
    }else{
        return;
    }
}

void Service::onServerResponseReceived(QNetworkReply* reply) {
    if (reply->error() != QNetworkReply::NoError) {
        qDebug() << "Network error:" << reply->errorString();
        reply->deleteLater();
        return;
    }
    QByteArray response = reply->readAll();
    reply->deleteLater();
    qDebug() << "Received server response, size:" << response.size();
    dbManager->saveDataFromJson(response);
}

void Service::onUpdateTimerTimeout() {
    fetchUpdatesFromServer();
}

void Service::sendContentToClient(QLocalSocket* client) {
    QStringList videos = dbManager->getCurrentVideos();
    QJsonObject response;
    response["command"] = "content_list";
    QJsonArray videoArray;
    for (const QString& path : videos) {

```

```

        videoArray.append(path);
    }
    response["videos"] = videoArray;
    QJsonDocument doc(response);
    QByteArray data = doc.toJson(QJsonDocument::Compact);
    if (client->state() == QLocalSocket::ConnectedState) {
        client->write(data);
        client->flush();
        qDebug() << "Sent content list to client. Size:" << data.size() <<
"bytes";
        qDebug() << "Sent JSON to client:" << data;    }
    }
}

void Service::sendUserDataToClients(const QJsonObject& data) {
    QJsonObject cmd;
    cmd["command"] = "update_user_data";
    cmd["first_name"] = data["first_name"].toString();
    cmd["last_name"] = data["last_name"].toString();

    QJsonDocument doc(cmd);
    QByteArray json = doc.toJson();

    for (QLocalSocket *client : clients) {
        if (client->state() == QLocalSocket::ConnectedState) {
            client->write(json);
            client->flush();
        }
    }
}

void Service::saveDataUser() {
    dbManager->saveUserData(
        m_userModel->firstName(),
        m_userModel->lastName(),
        m_userModel->department(),
        m_userModel->departmentCode());
}

```


Исходный код серверной части.

category.py

```
from django.db import models
from django.urls import reverse

class Category(models.Model):
    title = models.CharField(unique=True)
    descriptions = models.CharField(blank=True)
    slug = models.SlugField(verbose_name="URL", max_length=255, unique=True)

    def __str__(self):
        return self.title

    def get_absolute_url(self):
        return reverse('category', kwargs={"slug": self.slug})

    class Meta:
        ordering = ["title"]
        verbose_name = "Категория"
        verbose_name_plural = "Категории"
```

content.py

```
from datetime import date
from django.db import models
from .category import Category
from .scheduleContent import ScheduleContent

class Content(models.Model):
    PROCESSING_STATUS = (
        ('pending', 'В ожидании'),
        ('processing', 'В обработке'),
        ('completed', 'Завершено'),
        ('failed', 'Ошибка'),
    )

    title = models.CharField(max_length=255)
    path = models.FileField(upload_to=f"media/%Y/%m/%d/")
```

```

playOrder = models.IntegerField(db_index=True, null=True)
category = models.ForeignKey(Category, on_delete=models.CASCADE,
related_name='category_content')
pubDate=models.DateTimeField('Date published',auto_now_add=True)
schedule = models.ForeignKey(ScheduleContent,
on_delete=models.SET_DEFAULT, default="unknown", blank=True)
startDay = models.DateField(default=date.today(),verbose_name="Дата
начала")
endDay = models.DateField(default=date.today(),verbose_name="Дата
окончания")
processed_file = models.FileField(upload_to=f"processed/%Y/%m/%d/",
blank=True, null=True)
processing_status = models.CharField(
    max_length=20,
    choices=PROCESSING_STATUS,
    default='pending'
)

def save(self, *args, **kwargs):
    if not self.pk:
        super().save(*args, **kwargs)
        self.playOrder = self.id
        kwargs['force_insert'] = False
        super().save(*args, **kwargs)

def __str__(self):
    return self.title

class Meta:
    verbose_name = "Контент"
    verbose_name_plural = "Контент"

```

department.py

```

import random

from django.db import models
from django.urls import reverse
from .playList import Playlist

class Department(models.Model):
    title = models.CharField(unique=True)

```

```

code =models.CharField(max_length=50, unique=True)

playLists = models.ManyToManyField(Playlist, blank=True,
related_name="department")

slug = models.SlugField(verbose_name="URL", max_length=255,
unique=True)

def get_absolute_url(self):
    return reverse('departament', kwargs={"slug": self.slug})

def __str__(self):
    return self.title

class Meta:
    verbose_name = "Отдел"
    verbose_name_plural = "Отделы"

```

employee.py

```

from django.db import models
from django.core.validators import MaxValueValidator, MinValueValidator
from .department import Department
from .loginClient import LoginClient

class Employee(models.Model):
    name = models.CharField()
    lastName = models.CharField(blank=True)
    code = models.CharField(blank=True)
    descriptions =models.CharField(blank=True)
    department = models.ForeignKey(Department, on_delete=models.PROTECT,
related_name="department_employee", null =True)
    photo = models.ImageField(verbose_name="Фото",
upload_to=f"media/photo/%Y%m%d/", blank=True)
    login = models.ForeignKey(LoginClient, on_delete=models.SET_NULL,
null=True, blank=True, related_name="employee")
    slug = models.SlugField(verbose_name="URL", max_length=255, unique=True)
    rating = models.IntegerField(verbose_name="Рейтинг",
validators=[MinValueValidator(0), MaxValueValidator(5)], default=0 )

    def __str__(self):
        name = "".join([self.name, " ", self.lastName])
        return name or "unknown"

```

```

class Meta:
    verbose_name = "Сотрудник"
    verbose_name_plural = "Сотрудники"

```

loginClient.py

```

from django.db import models

```

```

class LoginClient(models.Model):
    loginMashine = models.CharField(default="unknown", null=True)
    ipaddress = models.CharField(blank=True, default="unknown", null=True)
    deviceId = models.CharField(blank=True, null=True)
    productId = models.CharField(blank=True, null=True)
    updatedAt = models.DateField(auto_now=True,
editable=True, verbose_name="Дата обращения")

```

```

def __str__(self):
    return self.loginMashine or "unknown"

```

```

class Meta:
    verbose_name = "Локальная учетная запись"
    verbose_name_plural = "Локальные учетные записи"

```

playList.py

```

from django.db import models
from django.urls import reverse

```

```

from .content import Content
from .tag import Tag

```

```

class Playlist(models.Model):
    name = models.CharField(max_length=255)
    playOrder = models.IntegerField(null=True)
    content = models.ManyToManyField(Content, blank=True,
related_name="playlist")
    tag = models.ManyToManyField(Tag, blank=True, related_name="playlist")
    slug = models.SlugField(verbose_name="URL", max_length=255, unique=True)

    def __str__(self):

```

```

        return self.name

    def get_absolute_url(self):
        return reverse('playlist', kwargs={"slug": self.slug})

    class Meta:
        verbose_name = "Плейлист"
        verbose_name_plural = "Плейлисты"

```

scheduleContent.py

```

from django.contrib.postgres.fields import ArrayField
from django.db import models

```

```

class ScheduleContent(models.Model):
    class Weekday(models.TextChoices):
        mon = 0, "Monday"
        tue = 1, "Tuesday"
        wed = 2, "Wednesday"
        thu = 3, "Thursday"
        fri = 4, "Friday"
        sut = 5, "Saturday"
        sun = 6, "Sunday"

    title = models.CharField(max_length=100)
    startTime = models.TimeField()
    endTime = models.TimeField()
    weekdays = ArrayField(
        models.CharField(choices=Weekday.choices),
        blank=True,
        default=list
    )
    def __str__(self):
        return self.title

    class Meta:
        verbose_name = "Расписание"
        verbose_name_plural = "Расписание"

```

views.py

```
import json
import os

from django.db.models import Exists, OuterRef
from django.http import JsonResponse, HttpResponse
from django.shortcuts import get_object_or_404, render, redirect
from django.urls import reverse
from django.views import View
from django.views.generic import ListView, DetailView, CreateView
from .models import Playlist, Department, Category, Content, Tag
from .forms import ContentUploadForm, AddCategoryForm, AddDepartmentForm,
AddPlaylistForm, UserAuthForm
from django.contrib.auth import login, logout
from django.contrib.auth.mixins import LoginRequiredMixin
from hrscreenapp.tasks import process_video

def user_login(request):
    if request.method == 'POST':
        form = UserAuthForm(data=request.POST)
        if form.is_valid():
            user = form.get_user()
            login(request, user)
            return redirect("index")
    else:
        form = UserAuthForm()
    return render(request, "hrscreenapp/login.html", {"form": form})

def user_logout(request):
    logout(request)
    return redirect('login')

class Home(LoginRequiredMixin, ListView):
    model = Playlist
    template_name = "hrscreenapp/index.html"
    context_object_name = "playlists"
    paginate_by = 8
    login_url = 'login'
    redirect_field_name = 'next'

    def get_context_data(self, *, object_list=None, **kwargs):
```

```

        context = super().get_context_data(**kwargs)
        context['title'] = "HR Screen"
        return context

class Search(LoginRequiredMixin, ListView):
    model = Playlist
    template_name = "hrscreenapp/index.html"
    context_object_name = "playlists"
    paginate_by = 8
    login_url = 'login'
    redirect_field_name = 'next'

    def get_context_data(self, *, object_list=None, **kwargs):
        context = super().get_context_data(**kwargs)
        context['title'] = "HR Screen"
        return context

    def get_queryset(self):
        return
        Playlist.objects.filter(name__icontains=self.request.GET.get('s'))

class GetCategory(DetailView):
    model = Category
    template_name = "hrscreenapp/category_content.html"
    context_object_name = "category"
    paginate_by = 8

    def get_context_data(self, *, object_list=None, **kwargs):
        context = super().get_context_data(**kwargs)
        context['title'] = "HR Screen"
        context['categorycontents'] = self.object.category_content.all()
        return context

class UploadContent(CreateView):
    form_class = ContentUploadForm
    template_name = "hrscreenapp/upload.html"

    def get_playlist(self):
        return get_object_or_404(Playlist, pk=self.kwargs['playlist_id'])

    def form_valid(self, form):
        video = form.save(commit=False)

```

```

        video.processing_status = 'pending'
        video.save()
        playlist = self.get_playlist()
        playlist.content.add(video)

        process_video.delay(video.id)
        playlist.content.add(video)

    return super().form_valid(form)

def get_context_data(self, **kwargs):
    context = super().get_context_data(**kwargs)
    context['title'] = "HR Screen"
    context['playlist'] = self.get_playlist()
    return context

def get_success_url(self):
    return reverse('playlist', kwargs={'slug': self.get_playlist().slug})

class AddCategory(CreateView):
    form_class = AddCategoryForm
    template_name = "hrscreenapp/addcategory.html"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context['title'] = "HR Screen"
        return context

class AddDepartment(CreateView):
    form_class = AddDepartmentForm
    template_name = "hrscreenapp/add-department.html"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context['title'] = "HR Screen"
        return context

class AddPlaylist(CreateView):

```



```

form_class = AddPlaylistForm
template_name = "hrscreenapp/add-playlist.html"

def get_context_data(self, **kwargs):
    context = super().get_context_data(**kwargs)
    context['title'] = "HR Screen"
    return context

class PlaylistByTag(ListView):
    model = Playlist
    template_name = "hrscreenapp/index.html"
    context_object_name = "playlists"
    paginate_by = 8

    def get_context_data(self, *, object_list=None, **kwargs):
        context = super().get_context_data(**kwargs)
        context['title'] = 'Записи по тегу : ' +
str(Tag.objects.get(slug=self.kwargs['slug']))

        return context

    def get_queryset(self):
        return Playlist.objects.filter(tag__slug= self.kwargs["slug"])

class GetPlaylist(DetailView):
    model = Playlist
    template_name = "hrscreenapp/playlist.html"
    context_object_name = "playlist"

    def get_context_data(self, *, object_list=None, **kwargs):
        context = super().get_context_data(**kwargs)
        context['title'] = "HR Screen"
        return context

class GetDepartament(DetailView):
    model = Department
    template_name = "hrscreenapp/departament.html"
    context_object_name = "departament"
    paginate_by = 8

```

```

def get_context_data(self, *, object_list=None, **kwargs):
    context = super().get_context_data(**kwargs)
    context['title'] = "HR Screen"
    context['employees'] = self.object.department_employee.all()
    return context

class PlaylistDepartmentsView(View):
    def get(self, request, pk):
        playlist = get_object_or_404(Playlist, pk=pk)
        departments = Department.objects.annotate(
            checked=Exists(playlist.department.filter(id=OuterRef('id')))
        ).values('id', 'title', 'checked')

        return JsonResponse({
            'departments': list(departments)
        })

class UpdatePlaylistDepartments(View):
    def post(self, request, pk):
        playlist = get_object_or_404(Playlist, pk=pk)
        try:
            data = json.loads(request.body)
            departments = data.get('departments', [])
            playlist.department.set(departments)
            departments_data = [
                {
                    'id': dept.id,
                    'title': dept.title,
                    'get_absolute_url': dept.get_absolute_url()
                }
                for dept in playlist.department.all()
            ]

            return JsonResponse({
                'departments': departments_data
            }, status=200)

        except json.JSONDecodeError:
            return JsonResponse({'error': 'Invalid JSON'}, status=400)
        except Exception as e:
            return JsonResponse({'error': str(e)}, status=500)

```

forms.py

```
import os
from django import forms
from django.core.exceptions import ValidationError
from django.forms import Select, TextInput, DateTimeInput, FileInput,
Textarea, PasswordInput
from django.contrib.auth.forms import AuthenticationForm
from slugify import slugify
from .models import Content, Category, ScheduleContent, Department, Playlist

class ContentUploadForm(forms.ModelForm):
    @staticmethod
    def validate_video_extension(value):
        ext = os.path.splitext(value.name)[1].lower()
        valid_extensions = ['.mp4', '.avi', '.m4v']

        if not ext in valid_extensions:
            raise ValidationError(
                'Допустимы только файлы форматов MP4 и AVI.'
            )

    category = forms.ModelChoiceField(
        queryset=Category.objects.all(),
        empty_label="Выберите категорию",
        label="Категория",
        widget=Select(attrs={"class": "form-control"}))

    schedule = forms.ModelChoiceField(
        queryset=ScheduleContent.objects.all(),
        empty_label="Выберите расписание",
        label="Расписание трансляции",
        widget=Select(attrs={"class": "form-control"}))

    playOrder = forms.IntegerField(
        required=False,
        widget=forms.HiddenInput()
    )

    path = forms.FileField(
        label="Файл",
        widget=FileInput(attrs={"class": "form-control"}),
```

```

        validators=[validate_video_extension]
    )

class Meta:
    model = Content
    fields = ['title', 'path', 'category', 'schedule',"startDay",
"endDay"]

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        if self.instance.pk:
            self.initial['playOrder'] = self.instance.id

    def save(self, commit=True):
        instance = super().save(commit=False)
        if not instance.pk:
            instance.save()
            instance.playOrder = instance.id
            instance.processing_status = 'pending'
            if commit:
                instance.save()
        return instance

    labels = {
        "title": "Название",
        "schedule": "Расписание",
        "path": "Файл",
        "startDay": "Дата начала",
        "endDay": "Дата окончания"
    }

    widgets = {
        "title": TextInput(attrs={"class": "form-control"}),
        "startDay": DateTimeInput(attrs={
            "class": "form-control",
            "type": "date"
        }),
        "endDay": DateTimeInput(attrs={
            "class": "form-control",
            "type": "date"
        }),
    }

```

```

        "path": FileInput(attrs={"class": "form-control"}),
    }

class AddCategoryForm(forms.ModelForm):
    slug = forms.CharField(
        required=False,
        widget=forms.HiddenInput()
    )

    def save(self, commit=True):
        instance = super().save(commit=False)
        if not instance.slug:
            base_slug = slugify(
                instance.title,
                separator='-',
                lowercase=True,
                replacements=[('ё', 'e')]
            )
            unique_slug = base_slug
            counter = 1

            while Category.objects.filter(slug=unique_slug).exists():
                unique_slug = f"{base_slug}-{counter}"
                counter += 1

            instance.slug = unique_slug

        if commit:
            instance.save()
        return instance

class Meta:
    model = Category
    fields = ['title', 'descriptions', 'slug']

    labels = {
        "title": "Название",
        "descriptions": "Описание",
    }

    widgets = {

```

```

        "title": TextInput(attrs={"class": "form-control"}),
        "descriptions": Textarea(attrs={
            "class": "form-control",
            "rows": 4,
            "placeholder": "Добавьте описание категории"
        }),
    }
}

```

```

class AddDepartmentForm(forms.ModelForm):
    slug = forms.CharField(
        required=False,
        widget=forms.HiddenInput()
    )

    def save(self, commit=True):
        instance = super().save(commit=False)

        if not instance.slug:
            base_slug = slugify(
                instance.title,
                separator='-',
                lowercase=True,
                replacements=[('ë', 'e')]
            )
            unique_slug = base_slug
            counter = 1

            while Department.objects.filter(slug=unique_slug).exists():
                unique_slug = f"{base_slug}-{counter}"
                counter += 1

            instance.slug = unique_slug

        if commit:
            instance.save()
        return instance

class Meta:
    model = Department

```

```

fields = ['title', 'code', 'slug']

labels = {
    "title": "Название",
    "code": "Код подразделения",
}

widgets = {
    "title": TextInput(attrs={"class": "form-control"}),
    "code": TextInput(attrs={"class": "form-control"}),
}

class AddPlaylistForm(forms.ModelForm):
    slug = forms.CharField(
        required=False,
        widget=forms.HiddenInput()
    )

    playOrder = forms.IntegerField(
        required=False,
        widget=forms.HiddenInput()
    )

class Meta:
    model = Playlist
    fields = ['name', 'playOrder', 'slug']
    labels = {
        "name": "Название"
    }
    widgets = {
        "name": forms.TextInput(attrs={"class": "form-control"}),
    }

def __init__(self, *args, **kwargs):
    super().__init__(*args, **kwargs)
    if self.instance.pk:
        self.initial['playOrder'] = self.instance.id

def save(self, commit=True):
    instance = super().save(commit=False)

```

```

        if not instance.slug:
            instance.slug = slugify(
                instance.name,
                separator='-',
                lowercase=True,
                replacements=[('ë', 'e')]
            )

        counter = 1
        original_slug = instance.slug

        while Playlist.objects.filter(slug=instance.slug).exists():
            instance.slug = f"{original_slug}-{counter}"
            counter += 1

        if not instance.pk:
            instance.save()
            instance.playOrder = instance.id

        if commit:
            instance.save()

    return instance

class UserAuthForm(AuthenticationForm):
    username = forms.CharField(label = "Имя пользователя",
        widget=TextInput(attrs={"class": "form-control"}))
    password = forms.CharField(label= "Пароль",
        widget=PasswordInput(attrs={"class": "form-control"}))

```

api/views.py

```

import uuid

from django.db.models import Prefetch
from rest_framework import viewsets
from rest_framework.authentication import SessionAuthentication
from slugify import slugify

from .serializer import DepartmentSerializer, GetDepartmentSerializer
from hrscreenapp.models import Department, Playlist, Content, Employee,
LoginClient
from rest_framework.permissions import IsAuthenticated

```



```

from .authentication import TokenAuthentication

class DepartmentPlaylistViewSet(viewsets.ModelViewSet):
    serializer_class = DepartmentSerializer
    authentication_classes = [TokenAuthentication, SessionAuthentication]
    permission_classes = [IsAuthenticated]

    def get_queryset(self):
        code = self.request.query_params.get("code")
        login = self.request.GET.get('login')
        ip = self.request.GET.get('ip')
        device_id = self.request.GET.get('deviceId')
        firstname = self.request.GET.get('firstname')
        lastname = self.request.GET.get('lastname')

        login_client, _ = LoginClient.objects.update_or_create(
            deviceId=device_id,
            defaults={
                'loginMashine': login,
                'ipaddress': ip
            }
        )

        if firstname and lastname:
            department = Department.objects.get(code=code)
            slug_base = slugify(f"{firstname}-{lastname}")
            unique_slug = f"{slug_base}-{uuid.uuid4().hex[:8]}"
            Employee.objects.update_or_create(
                login=login_client,
                department=department,
                defaults={
                    'name': firstname,
                    'lastName': lastname,
                    'slug': unique_slug, })

        if not code:
            return Department.objects.none()

        return Department.objects.filter(code=code).prefetch_related(
            Prefetch(
                "playLists",
                queryset=Playlist.objects.order_by("playOrder")
                .prefetch_related(

```

```

        Prefetch(
            "content",
            queryset=Content.objects.order_by("playOrder")
        )
    )
)
)

```

```

class GetDepartmentViewSet(viewsets.ModelViewSet):
    serializer_class = GetDepartmentSerializer
    queryset = Department.objects.all()
    authentication_classes = [TokenAuthentication, SessionAuthentication]
    permission_classes = [IsAuthenticated]

```

authentication.py

```

from rest_framework.authentication import BaseAuthentication
from rest_framework.exceptions import AuthenticationFailed
from hrscreenapp.models.token import Token

class TokenAuthentication(BaseAuthentication):
    def authenticate(self, request):
        token = request.META.get('HTTP_AUTHORIZATION')
        if not token:
            return None

        try:
            token = token.split(' ')[1]
        except IndexError:
            raise AuthenticationFailed('Неверный формат токена.')

        try:
            token_obj = Token.objects.get(token=token)
            user = token_obj.user
        except Token.DoesNotExist:
            raise AuthenticationFailed('Токен не найден.')

        return (user, token_obj)

```

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования
«Санкт-Петербургский государственный университет
аэрокосмического приборостроения»

ОТЗЫВ РУКОВОДИТЕЛЯ
НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ

на тему Разработка системы корпоративного телевидения для организаций

выполненную студентом группы № Z0431

Кузьминым Иваном Александровичем

фамилия, имя, отчество студента

по направлению подготовки / 09.03.04
специальности (код)

Программная инженерия
(наименование направления подготовки/специальности)

Актуальность темы работы:

Крупные компании используют различные методики повышения вовлеченности сотрудников, развития корпоративной культуры, быстрой адаптации новичков. Кузьмин И.А. разработал систему корпоративного телевидения для организаций, как средство распространения мультимедийного корпоративного контента среди сотрудников.

Цель и задачи работы:

Разработка цифровой платформы для организации корпоративного телевидения, которая будет способствовать улучшению внутренней коммуникации в компании и оптимизации управления видеоконтентом.

Общая оценка выполнения поставленной перед студентом задачи, основные достоинства и недостатки работы:

В ходе выполнения квалификационной работы Кузьмин И.А. провел исследование предметной области, выполнил сравнение с аналогами, спроектировал и разработал цифровую платформу корпоративного телевидения, позволяющую организовать централизованное управление медиаконтентом, настраивать трансляцию по подразделениям, отображать видеоконтент на заставке мониторов в офисах организации. Для реализации цифровой платформы Кузьмин И.А. использовал язык программирования Python, фреймворк Django, систему управления базами данных (СУБД) PostgreSQL, NoSQL базу данных Redis. Клиентская часть создана с использованием кроссплатформенного фреймворка Qt6 и СУБД SQLite.

Степень самостоятельности и способности к исследовательской работе студента:
В ходе работы Кузьмин И.А. проявил способности к исследовательской работе, умение работы с различными источниками информации, навыки поиска и обобщения материала, необходимого для реализации системы, формулирования выводов.

Проверка текста выпускной квалификационной работы с использованием системы antiplagiat, проводившаяся 11.06.2025, показывает оригинальность содержания на уровне 94.09%. Текст пояснительной записки оформлен грамотно, изложение последовательное.

Оценка деятельности студента в период подготовки выпускной квалификационной работы:

Кузьмин И.А. проявил добросовестность, ответственность и аккуратность при работе над квалификационной работой.

Общий вывод:

В ходе квалификационной работы Кузьмин И.А. разработал цифровую платформу для организации корпоративного телевидения. Кузьмин И.А. решил сложную задачу, практически значимую задачу, заслуживает присвоения квалификации бакалавра по направлению 09.03.04.

В работе не содержится информация с ограниченным доступом и отсутствуют сведения, представляющие коммерческую ценность.

Руководитель

доц., канд. тех. наук.

должность, уч. степень, звание

 06.06.25

подпись, дата

А.В. Фомин

инициалы, фамилия



Разработка системы корпоративного телевидения для организаций

Кузьмин Иван Александрович, Z0431

Фомин Александр Владимирович, доцент, канд. техн. наук

2025

Разработка системы корпоративного телевидения на основе современных веб- и мультимедийных технологий для повышения эффективности внутренних коммуникаций в организации.

1. Проблемы:

- *Существующие каналы корпоративных коммуникаций (email-рассылки, порталы, собрания) демонстрируют низкую эффективность*

2. Подходы:

- *Персонализация контента на основе роли сотрудника*

3. Внедрение системы:

- *Увеличить вовлеченность сотрудников*
- *Сократить время доведения информации*
- *Снизить затраты на адаптацию персонала*



Задачи работы:

1.Реализовать централизованное управление контентом

- Загрузка, редактирование и удаление контента с сервера.
- Поддержка актуальности информации.

2.Разработать систему авторизации пользователей

- Безопасный вход с разных устройств.

3.Создать удобный интерфейс для просмотра контента

- Доступ к видеоматериалам.

4.Обеспечить настройку ротации контента

- Гибкое расписание трансляции по дням недели и времени.

5.Добавить поиск и фильтрацию контента

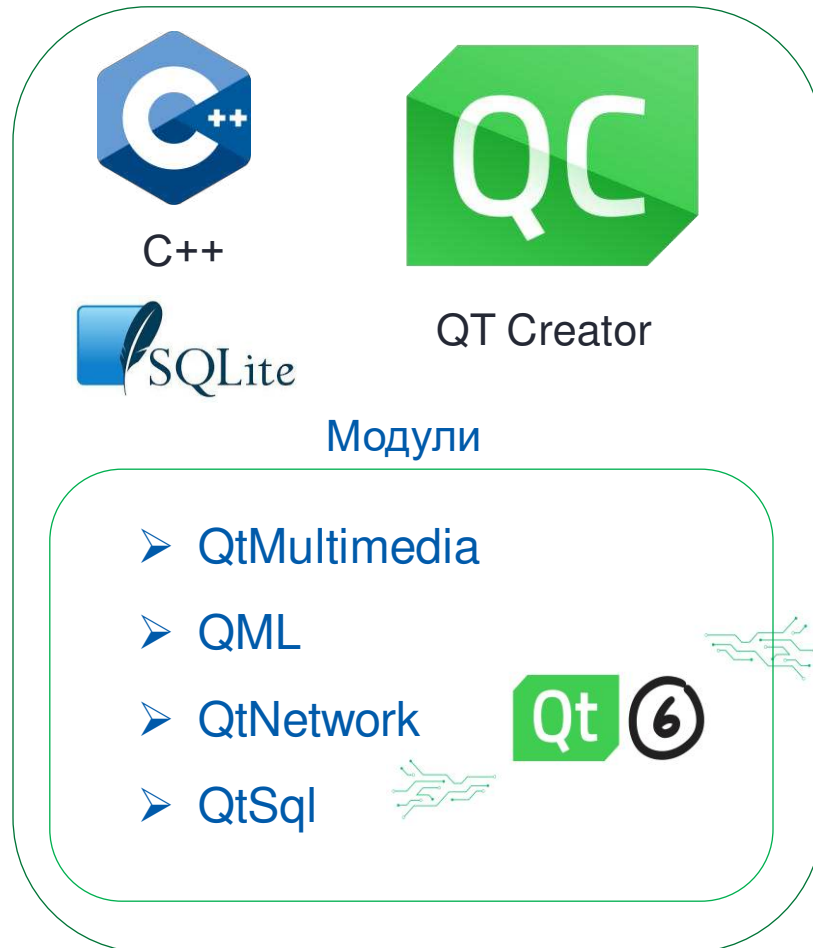
- Поиск по тематике, дате загрузки и другим критериям.

6.Разработать Windows приложение

- Автоматическое воспроизведение контента в режиме ожидания.



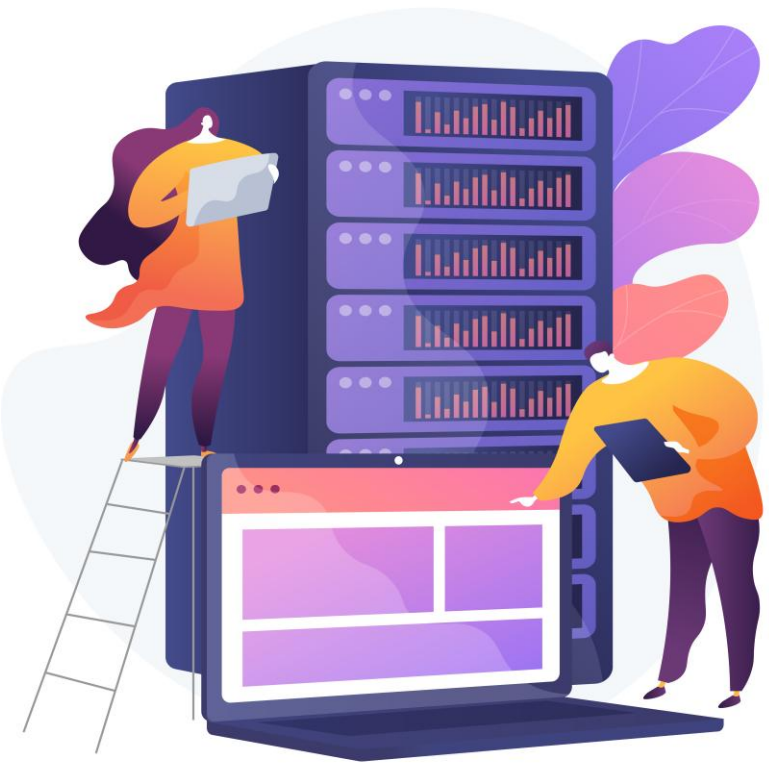
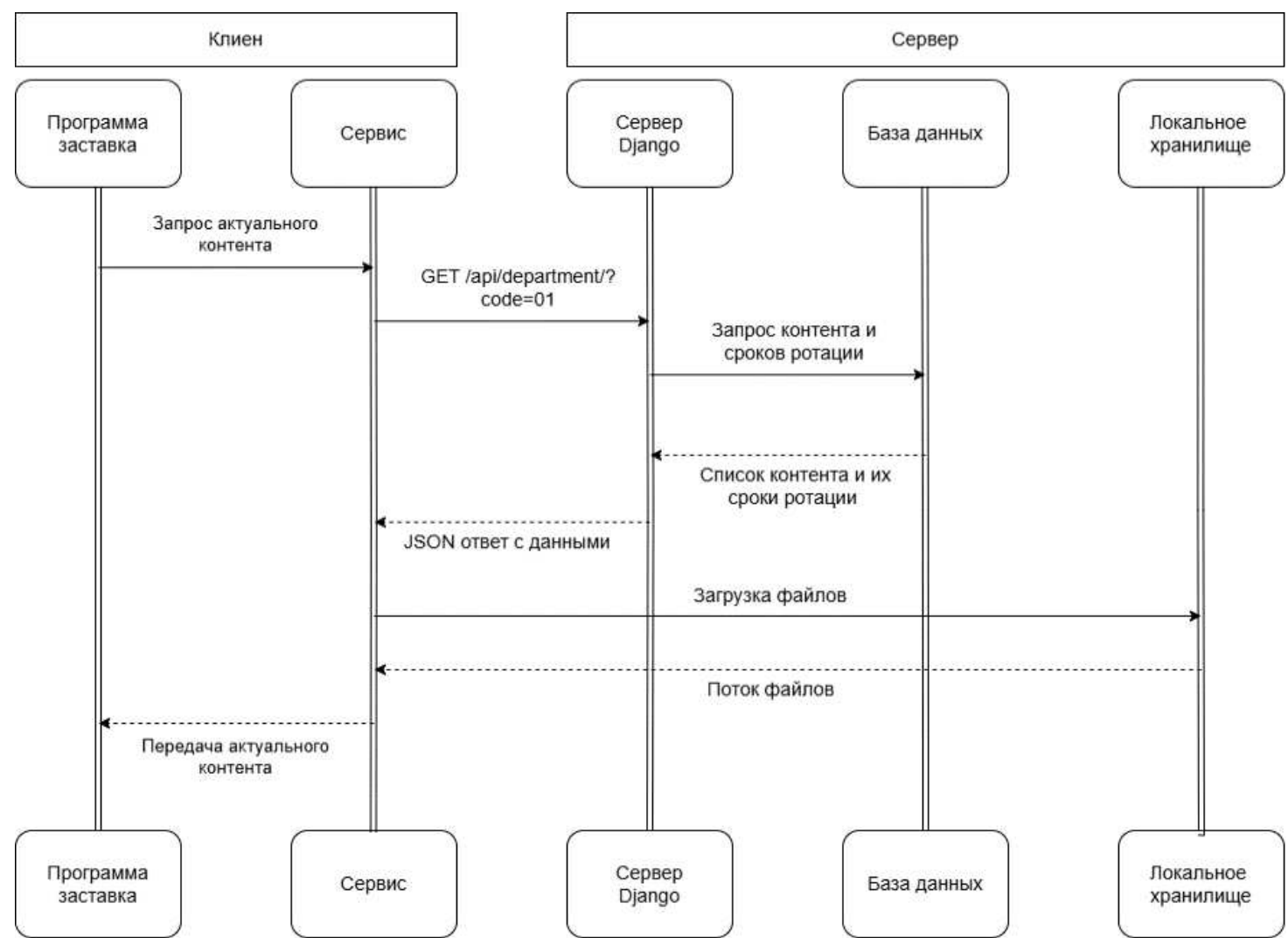
Windows приложение

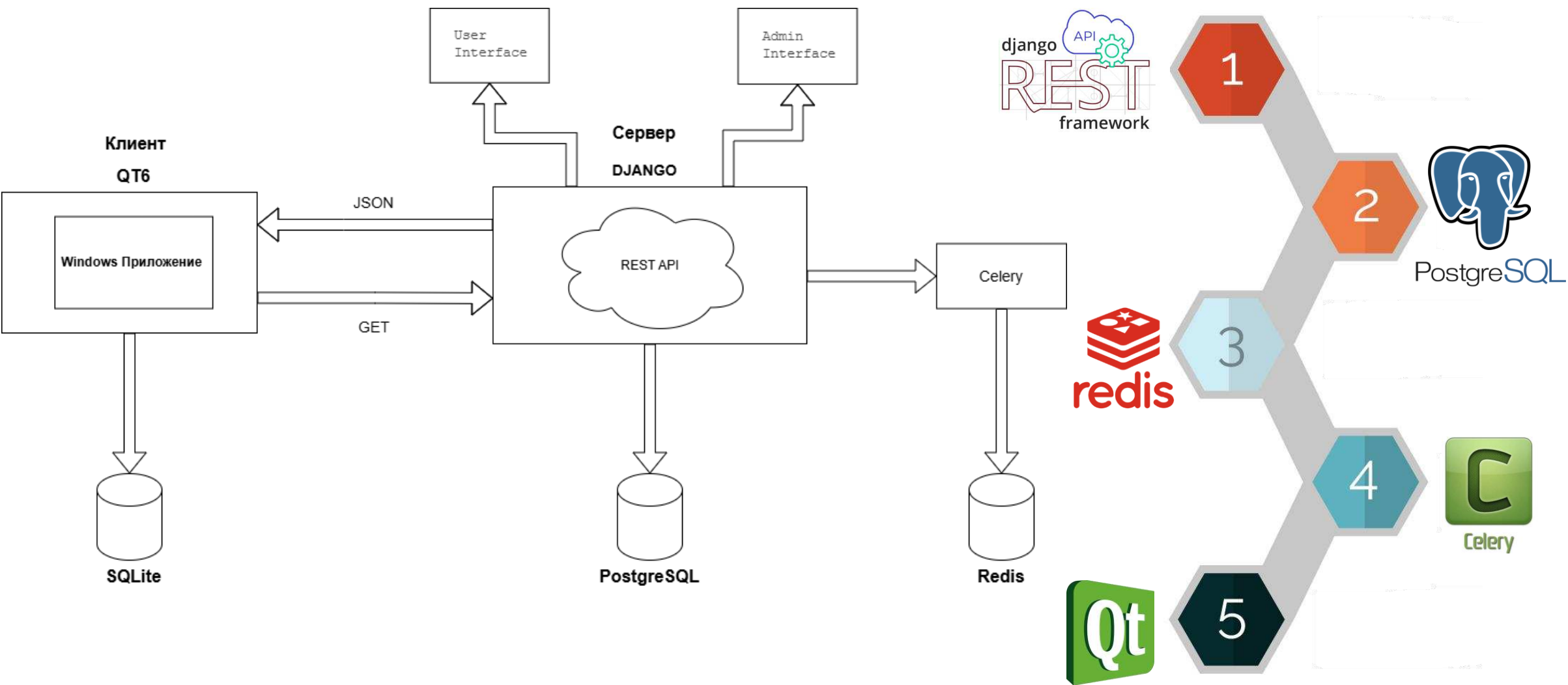


Серверная часть

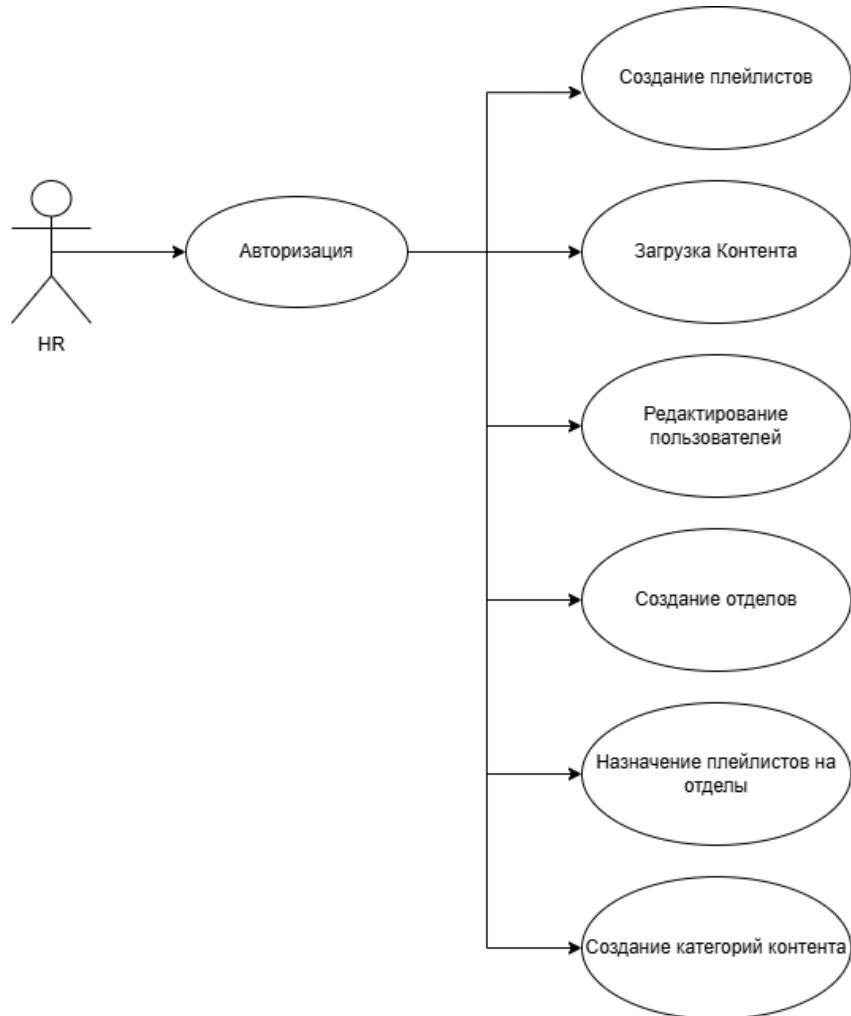


Архитектура. Диаграмма последовательности





Серверная часть: панель управления контентом



Клиентская часть: Windows приложение

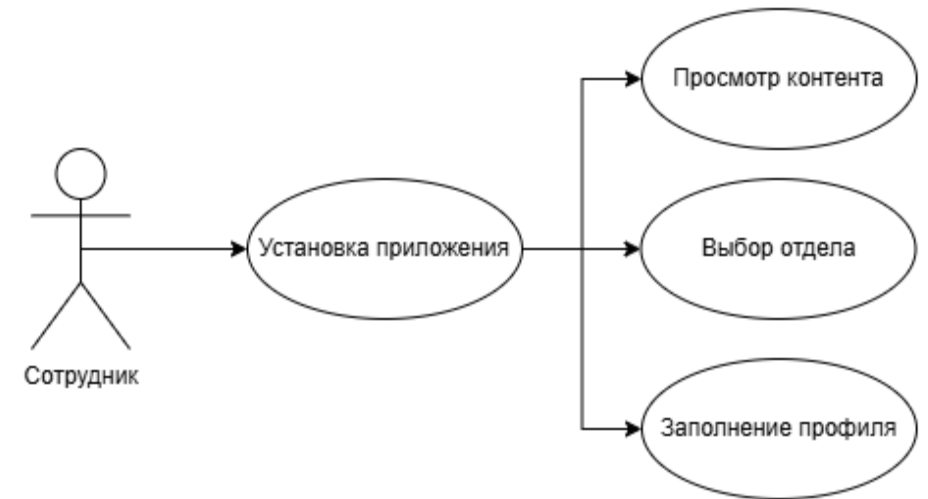
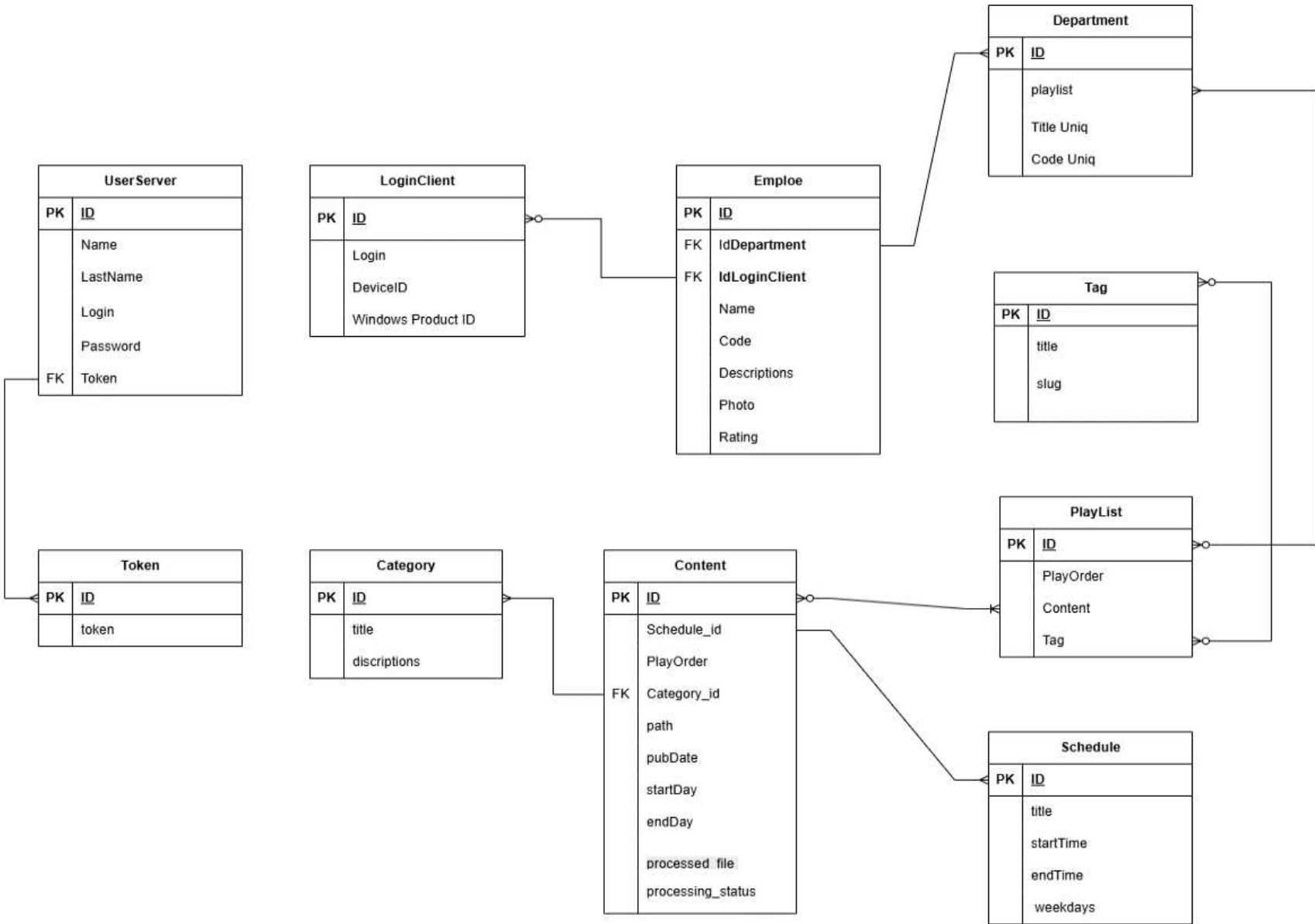


Диаграмма «Сущность-связь» базы данных



Добро пожаловать, lumpy

Меню

Плейлисты

Отделы

Бухгалтерия

Техническая поддержка

Маркетинг

HR
Screen

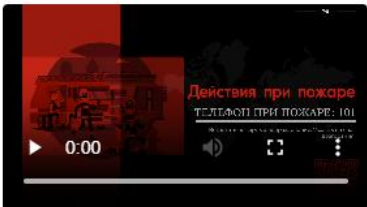
Добавить плейлист



Пожарная безопасность

Загрузить

Бухгалтерия Техническая поддержка Маркетинг



[Открыть >](#)



Общий

Загрузить

Бухгалтерия Техническая поддержка Маркетинг



[Открыть >](#)

Поиск



Категории контента

Вакансии	(3)
Корпоративная культура	(2)
Мотивация	(0)
Общая	(3)
Пожарная безопасность	(2)

Лучшие сотрудники



Иван Иванов
 June 28, 2019 5



Ivan Kuzmin
 June 28, 2019 5



Кузьмин Иван
 June 28, 2019 4

Результаты. Назначение плейлистов по отделам



Добро пожаловать, lumpy

Меню

Плейлисты

Отделы

Бухгалтерия

Техническая поддержка

Маркетинг



Добавить плейлист

Добавить отделы к плейлисту

☒ Бухгалтерия

☒ Техническая поддержка

☒ Маркетинг

Сохранить

Отмена

Открыть >

Общий

Загрузить

Бухгалтерия

Техническая поддержка

Маркетинг

Погода: 15°C

Планёрка

0:00

0:00

Открыть >

Поиск

Категории контента

Вакансии

Корпоративная культура

Мотивация

Общая

Пожарная безопасность

Лучшие сотрудники

Иван Иванов

Ivan Kuzmin

Кузьмин Иван

7/8



Действия при пожаре

ТЕЛЕФОН ПРИ ПОЖАРЕ: 101

Встретить пожарные подразделения. Указать на очаг возгорания.



* ПОСЛЕ ОЗНАКОМЛЕНИЯ С ИНФОРМАЦИЕЙ ПЕРЕЙДИТЕ ПО ССЫЛКЕ ДЛЯ ПРОХОЖДЕНИЯ ТЕСТИРОВАНИЯ

Результаты. Список обращений пользователей к системе



Добро пожаловать, lumpry

Меню

Плейлисты

Отделы +

Бухгалтерия

Техническая поддержка

Маркетинг

- 

Екатерина Иванова
Рейтинг: 0
Личный номер:
Логин: katerina
IP Адрес: 10.90.0.4
Дата последнего обращения 3 июня 2025 г.
- 

Андрей Петров
Рейтинг: 0
Личный номер: 022
Логин: super
IP Адрес: 10.90.0.4
Дата последнего обращения 3 июня 2025 г.
- 

Иван Кузьмин
Рейтинг: 0
Личный номер:
Логин: super
IP Адрес: 10.90.0.4
Дата последнего обращения 3 июня 2025 г.

Корпоративная культура (2)

Мотивация (0)

Общая (3)

Пожарная безопасность (2)

Лучшие сотрудники



Иван Иванов
June 28, 2019 5



Ivan Kuzmin
June 28, 2019 5



Кузьмин Иван
June 28, 2019 4



Екатерина Иванова
June 28, 2019 0

Разработка системы корпоративного телевидения, представленная в данной практической работе, является ответом на актуальные вызовы современных корпоративных коммуникаций. Особое внимание было уделено созданию системы, которая не просто транслирует контент, но и формирует полноценную медиасреду для взаимодействия между всеми участниками корпоративных процессов.

Платформа сочетает надежность серверных решений (Django, PostgreSQL) с современными возможностями клиентского приложения (Qt6). Разработанная архитектура демонстрирует отличную масштабируемость - от небольших компаний до крупных корпораций с распределенной структурой и предусматривает несколько направлений для дальнейшего развития:

- Развитие инструментов интерактивности (квизы, опросы, голосования),
- Интеграция с корпоративными системами управления знаниями,
- Разработка мобильной версии с поддержкой push-уведомлений.



Таким образом, система корпоративного телевидения представляет собой не просто инструмент, а целую экосистему для построения эффективных коммуникаций в организациях любой отрасли и масштаба.



ГУАП

<https://guap.ru>



Спасибо за внимание!